

CS5351 软件工程 2025-2026 学年第一学期

软件过程模型

黄志安教授

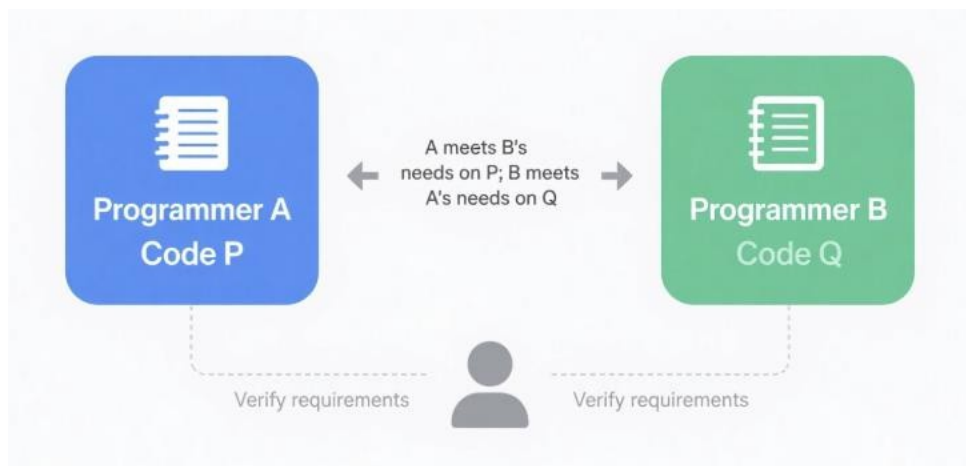
邮箱: huang.za@cityu-dg.edu.cn

课程目标

- ◆ 一支由**实力强劲且全情投入**的程序员组成的团队（设为 $G = \{\text{程序员A}, \text{程序员B}, \text{程序员C}\}$ ），能够协作开发代码项目，最终向用户交付应用程序
B}) 可通过协作开发交付应用程序。

- A对代码P及其职责了如指掌
- B对代码Q及其职责了如指掌
- A能交付代码P，B能交付代码Q
- A满足B在P方面的需求；B满足A在Q方面的需求
- A和B可随时向用户U核实具体需求

- ◆ 完美案例。



动机（现实情况）

- ◆ 一群普通且情绪化时有的程序员（设 $G = \{\text{程序员}A, \text{程序员}B\}$ ）可通过代码项目开发向用户交付应用程序。
 - A 熟悉代码 P 及其职责
逐步掌握
 - B 掌握代码 Q 及其职责
逐步掌握
 - A 可能交付功能不全**且存在缺陷**的 P ， B 可能交付功能不全**且存在缺陷**的 Q
 - A **可能不清楚** B 对 P 的**确切需求**； B **可能不清楚** A 对 Q 的**确切需求**。由于 B 对 Q 的认知是渐进的， B 对 P 的需求会随时间变化。
 P 随时间变化。
 - A 和 B **有时**可通过**用户** U 获取并验证精确需求；但**老用户** U **离职，新用户** U **接替**！部分用户**缺乏**业务领域经验。



动机（针对现实中大型项目）

假设第1个月，团队 $G = \{\text{程序员}A, \text{程序员}B\}$

- ◆ 第二个月， $G = \{\text{程序员}B\}$ ， A 已离职。
- ◆ 第3个月， $G = \{\text{程序员}A1, \text{程序员}B\}$ ， $A1$ 接替 A 的职位。
- ◆ 第4个月， $G = \{\text{程序员}A1\}$ ， B 辞职。
- ◆ 第5个月， $G = \{\text{程序员}A1, \text{程序员}B1\}$ ， $B1$ 接替 B 的职位。
- ◆ 第6个月， $G = \{\text{程序员}B1\}$ ， $A1$ 辞职。
- ◆ 第7个月， $G = \{\text{程序员}A2, \text{程序员}B1\}$ ， $A2$ 接替 $A1$ 的职位。

分析

- ◆ 第1月， A 与 B 会见 U 获取用户需求
- ◆ 在第7个月， $A2$ 通过 $A \Rightarrow B \Rightarrow A1 \Rightarrow B1 \Rightarrow A2$ 获取代码 P 的需求。

- A2能在多大程度上获取P的具体操作要求？
- A2能否与B1共同厘清P的需求？可能不完整，但总比毫无头绪强

动机（针对现实中的大型项目）

- ◆ A2希望了解
 - A数月前从U处收集的需求（标记为R）
 - A2接手前R的最新变更情况
 - P 集合中已识别并设计但尚未（完全）实现的函数集 $\{f_1, \dots, f_n\}$ ，针对最新需求而言
 - 针对P中已实现函数 $\{g_1, \dots, g_m\}$ 与最新需求相关的反馈（来自B或U）
 - P中已知且关键但尚未完全修复的问题
- ◆ B2希望尽量减少工作量，将P的信息传递给A2
 - 实际上，A1也希望尽量减少工作量，将Q的信息传递给B2
- ◆ U不希望A-A2和B-B1的工作影响U的日常事务。
 - 接待A/B仅为副业。
 - 你能指望像U这样的房产经纪人去见A-A2和B-B1，而不是见（因此无法获得佣金或薪资降低）？
- ◆ 人人都渴望快速获取参考信息并推进进程
- ◆ 当项目规模庞大时，无人能掌握全局细节。
 - 在执行任务前等待明确的指示和清晰的任务说明是不现实的。

需求

- ◆ 前几张幻灯片中的软件项目需要制定策略来控制项目任务的顺序，
 - 以降低生产间接成本，
 - 提升信息传递效率，
 - 实现从用户期望到交付成果的可追溯性。
 - 预测并追踪任务/项目完成情况及未决事项，
 - 提升代码质量，以及.....

◆ 否则.....[下一页]

缺陷与延误无处不在：

COMPUTERWORLD

UNITED STATES ▼

IDG TECH(TALK) COMMUNITY

WINDOWS

MOBILE

OFFICE SOFTWARE

APPLE

SHARK TANK

EVENTS

INSIDER

Home > Vertical Industries > Retail Industry

SLIDESHOW

Top software failures in recent history

The biggest software failures in recent history including ransomware attacks, IT outages and data leakages that have affected some of the biggest companies and millions of customers around the world

By Computerworld UK staff, Computerworld | Feb 17, 2020 6:20 am PST

February 2020: Heathrow disruption

More than 100 flights to and from London's Heathrow airport were disrupted on Sunday 16 February, 2020, after it was hit by technical issues affecting departure boards and check-in systems, leaving passengers with little information about their flights and limiting the use of electronic tickets.

August 2019: British Airways (again)

British Airways was struck by yet another IT glitch in August 2019, when system failures caused more than 100 flights to be cancelled and more than 200 others to be delayed.

Facebook, Instagram and WhatsApp

In the first week of July 2019, users across the globe found themselves unable to load photos in the Facebook News Feed, view stories on Instagram, or send messages in WhatsApp. Although Facebook didn't detail the reason for the outage, it did release a

Bitcoin Unlimited

Bitcoin Unlimited suffered a serious memory leak which caused several nodes to fall from 800 to about 300. This is almost 70 percent of the nodes run by Bitcoin Unlimited at the time.

HSBC suffers major outage

Less than a week into 2016, HSBC became the first bank to suffer a major IT outage. Millions of the bank's customers were unable to access online accounts. Services only returned to normal after a two-day outage.

缺陷与延误无处不在： 机场混乱与系统升级

◆ 香港机场

You are here: [silicon.com](#) > [Hardware](#) > [PDAs](#)

PDAs

Hong Kong airport systems grind to a halt

By Polly Raymond

Published: 9 July 1998 09:37 BST

Computer bugs have brought Hong Kong's airport cargo handling system to its knees.

Show related articles

The \$20bn (£12.2bn) airport was only opened on Monday but has quickly stumbled upon system problems, which have caused 24 hour delays for some travellers.

A spokeswoman for the Hong Kong Embassy said: "It's only a minor problem" but was unable to confirm whether systems have returned to normal.

The bugs are in the cargo IT systems and baggage handling operations. Cargo operations have been

Hong Kong's air traffic controllers left flying blind for six minutes as HK\$1.56 billion system malfunctions again – but authorities insist safety was not affected

Civil Aviation Department recognises 'importance of incident' as lawmaker Jeremy Tam questions delay in alerting public about the failure

PUBLISHED: Wednesday, 13 August, 2018, 8:45pm
UPDATED: Wednesday, 13 August, 2018, 11:21am

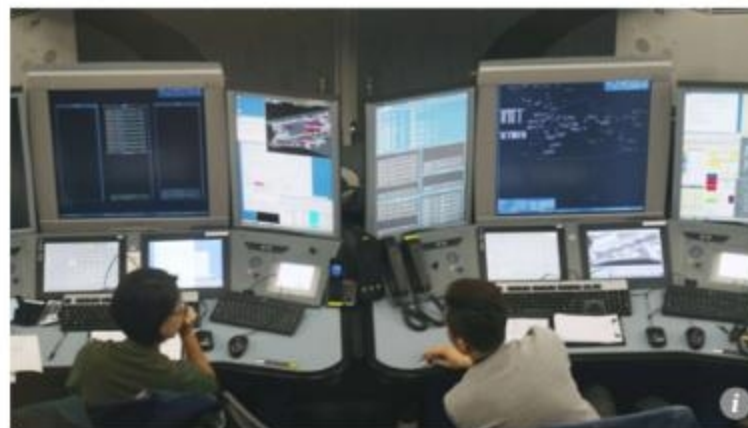
COMMENTS: 5

Hong Kong's air traffic control system suffers another glitch

Back-up system deployed for first time as operators lose information on positioning and altitude of planes

PUBLISHED: Saturday 08 April, 2017, 8:00pm
UPDATED: Saturday 08 April, 2017, 10:13pm

COMMENTS: 11



<https://www.scmp.com>

UK Airport Chaos Due To Software Glitch

By Spyros Georgilidakis | May 27, 2023



Passengers arriving at more than one UK airport faced severe delays because of software problems with air control systems.

港铁

Hong Kong / Hong Kong economy

Software error to blame for two-hour Hong Kong MTR stoppage that left tens of thousands stranded

Transport operator says problematic code has been fixed, and improved recovery procedures put in place



Danny Mok

+ FOLLOW

Published: 8:50am, 13 Mar, 2018

Why you can trust SCMP



HONG KONG

SIGNALLING BUG DELAYS MTR'S SHA TIN-CENTRAL EXTENSION MILESTONE

BY JAMES OCKENDEN ON SEPTEMBER 12, 2020

Signal system supplier confirms software issue after Central train crash, MTR Corp says



by KRIS CHENG

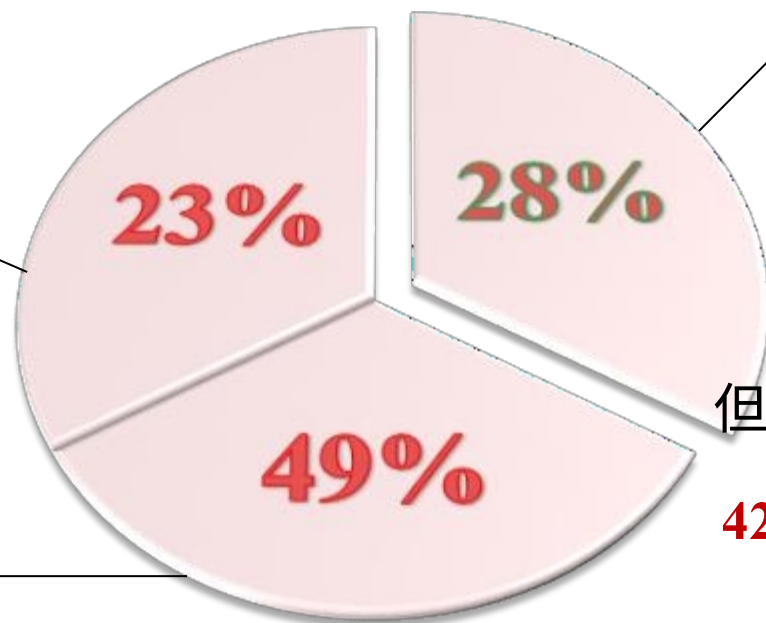
19:36, 18 MARCH 2019 Updated 22:58, 31 MARCH 2020



事实（数年前）： 软件开发项目

在完成前**被取消**。

超出原始预算
89%



按时完成**且**
未超预算。

但仅能实现原始需求的
42%。

问题

被取消的项目中没有任何东西能被利用或复用吗？

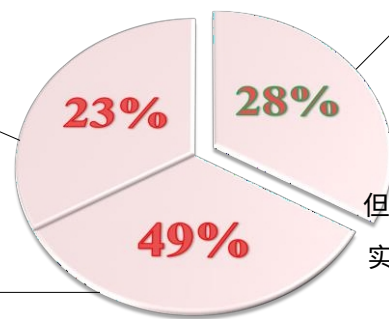
如何识别需求子集，

事实：

~~软件开发项目~~

在**项目**
完成前

成本超出原始预算
89%



按时且在预算
内完成。

但原始需求中仅有**42%**得以实现。

其中
在它们之间保持一致，并追踪其发展

如何优化活动流程以降低成本

？

N 需优先处理需求以指导开发工作

最新研究发现

◆ 斯坦迪什集团2020年度CHAOS报告

- 66%的项目以部分/完全失败告终。
 - 31%被取消
 - 53% 项目性能下降
- 大型项目：成功率10%
- 小型项目：成功率90%

◆ 麦肯锡2020年报告

- 17%的大型项目进展不顺

◆ 波士顿咨询公司（2020年）

- 70%的数字化转型努力未能达到预期目标。

现状2018年项目失败率

FEATURE

IT project success rates finally improving

After years of stagnating IT project success rates, a new survey from PMI shows that rates are finally on the rise.

<https://www.cio.com>

- ◆ 兰利指出，PMI研究还发现了一系列冠军企业的共同特征。研究表明，除了专注于**效益实现**外，冠军企业更倾向于**培养项目管理人才**，至少在组织内部**设立一个项目管理办公室（PMO）**，**推动高层赞助**，并**采用敏捷方法进行项目管理**。

软件工程过程

软件工程过程模型包含 (1) 活动 (2) 技术 (3) 可交付成果及 (4) 工具，旨在引导软件工程师与项目经理遵循 (5) **最佳实践**（如结对编程、敏捷方法）来运用(1)-(4) 项管理软件系统

SE流程 — 活动

- ◆ 收集用户需求
- ◆ 软件设计
- ◆ 编码/实现
- ◆ 测试
- ◆ 在用户站点部署软件
- ◆ 维护软件
- ◆ 配置管理
- ◆ 项目管理

示例化软件需求

用户需求 Set of Natural Language Requirement

Requirements - Train Station System

Develop a system to model the behavior of a Train-Station. You need to model a train entering the station from the north and then leaving the station to the south. A crossing with boom gates and flashing red lights is located just south of the station. There is a signal to the north of the station that only allows a train to enter when the station is not occupied, that is, when the north signal is green. There is also an exit signal light that ensures the train can only leave the station when the boom gates are down. There is also a north detector that can detect the train approaching the station region from the north. And, there is an exit detector that detects when a train leaves to the south. The following requirements capture the desired behavior.

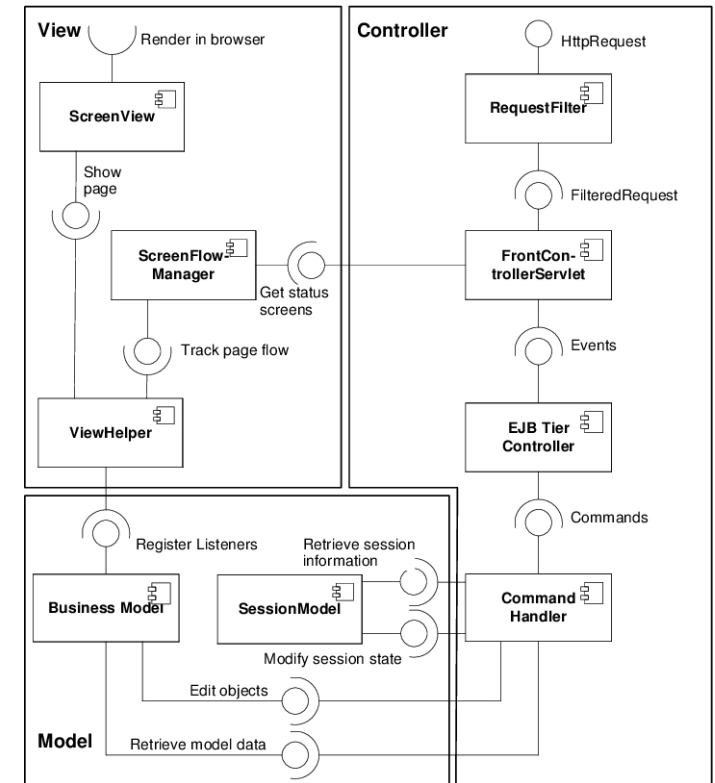
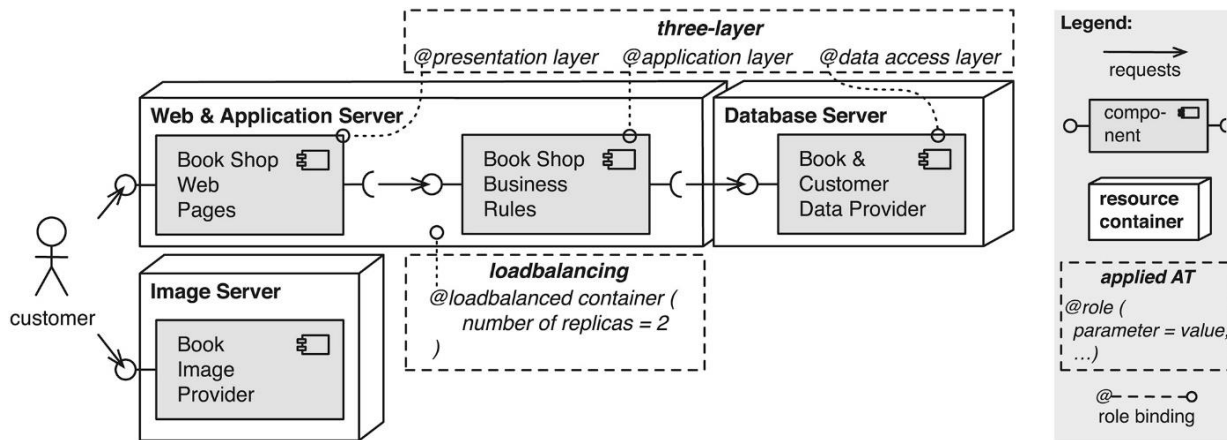
- R1.** Initially the station is not occupied. The north signal turns green whenever the station is not occupied. Whenever the north signal is green a train may approach from the north. When approaching from the north a train is detected, by the north detector, which causes the north signal to turn red.
- R2.** When the north detector detects a train it causes the crossing lights to start flashing red. At the same time, a timer starts timing and when it times out it causes the boom gates to be lowered after which the exit light turns green.
- R3.** After the train is detected the north detector, it subsequently arrives at the station, the doors open, the people disembark, and then the doors close.
- R4.** After the doors close the train may leave the station only when and if the exit light is green. When the train leaves the station, heading south, it is detected by the exit detector which means the station is again not occupied. This causes the north signal to turn green and the exit light to turn red. When the exit detector detects the train leaving, it also causes the boom gates to be raised and then the crossing lights to stop flashing red.

For the purposes of the exercise ignore trains approaching the station from the south. This additional requirement can be integrated later as a separate exercise. Also ignore situations where the train does not stop at the station - this too requires some refinements to the requirements.

非功能性需求

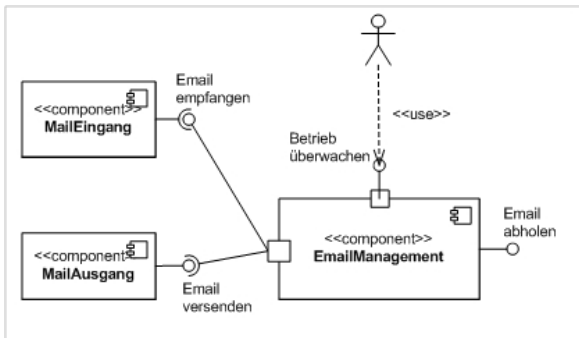
- Product requirement
 - 4.C.8 It shall be possible for all necessary communication between the APSE and the user to be expressed in the standard Ada character set
- Organisational requirement
 - 9.3.2 The system development process and deliverable documents shall conform to the process and deliverables defined in XYZCo-SP-STAN-95
- External requirement
 - 7.6.5 The system shall not disclose any personal information about customers apart from their name and reference number to the operators of the system

示例化软件架构

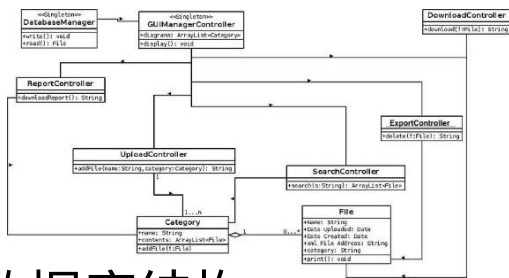


示例化软件设计

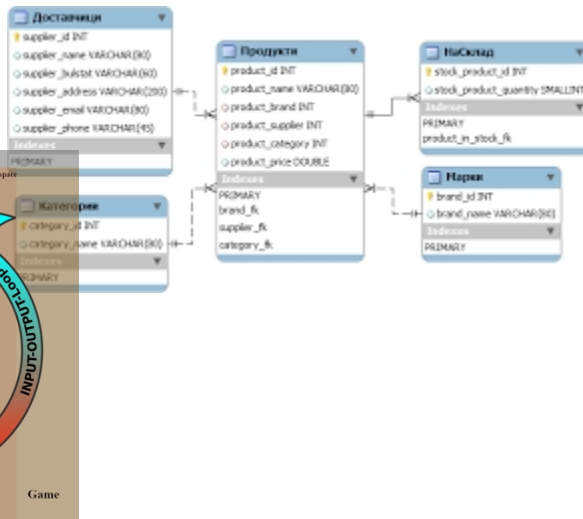
静态视图：组件图



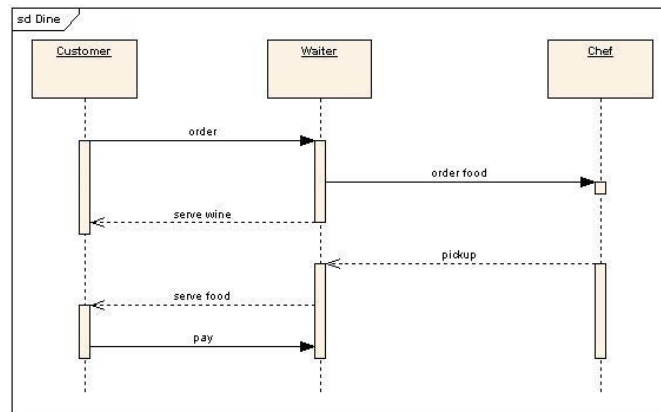
类图



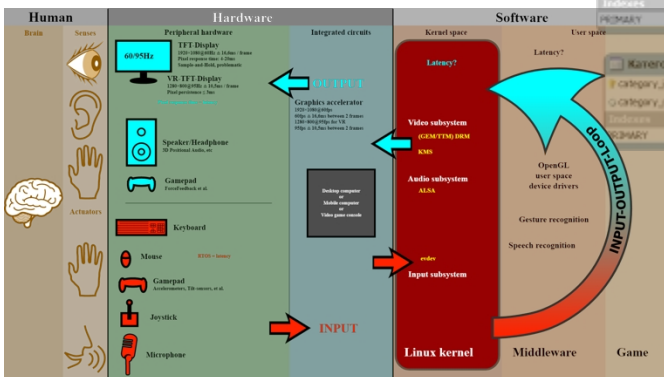
数据库结构



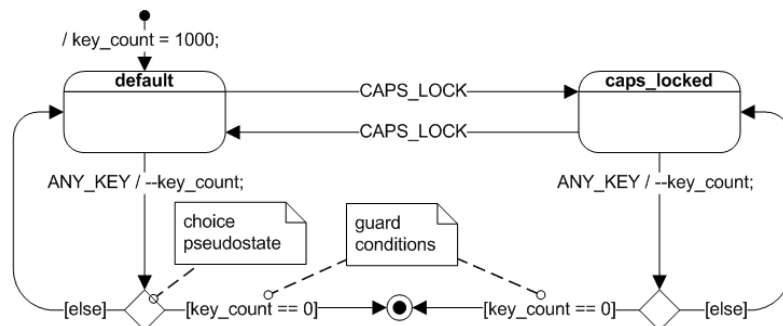
动态视图：序列图



用户界面设计

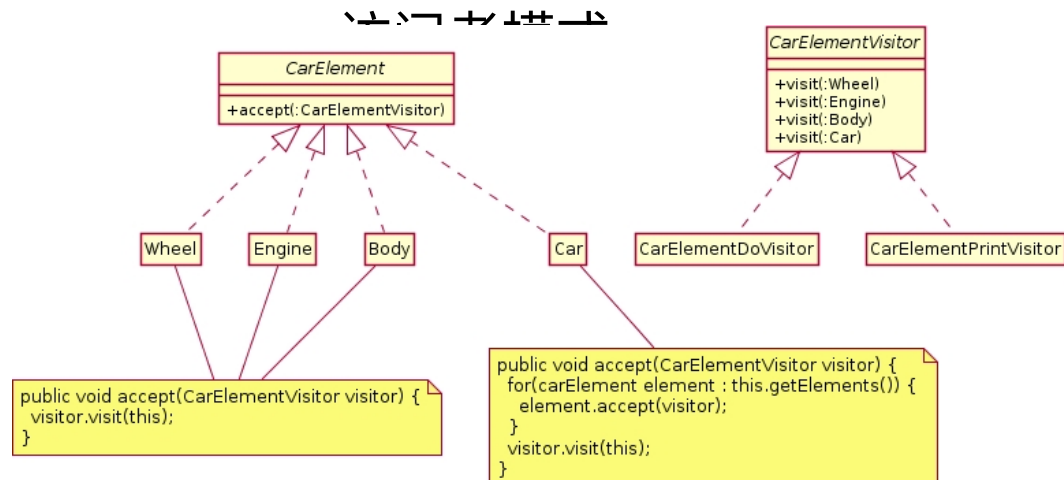
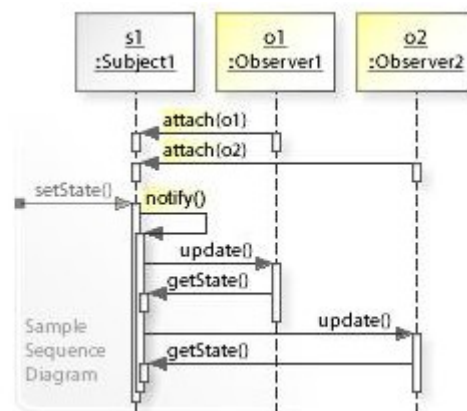
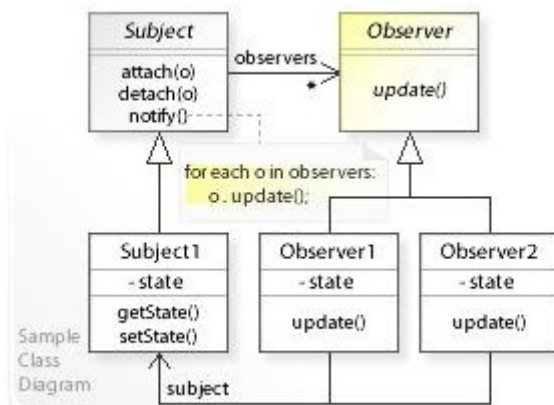


状态转换图



示例化软件设计模式

观察者模式



示例化编码

```
int risky_programming(char *input) {  
    char str[1000];  
  
    // ...  
  
    strcpy(str, input); // Copy input.  
  
    // ...  
}
```

不良编码

```
int secure_programming(char *input) {  
    char str[1000+1]; // One more for the null character.  
  
    // ...  
  
    // Copy input without exceeding the length of the destination.  
    strncpy(str, input, sizeof(str));  
  
    // If strlen(input) >= sizeof(str) then strncpy won't null terminate.  
    // We counter this by always setting the last character in the buffer to NUL,  
    // effectively cropping the string to the maximum length we can handle.  
    // One can also decide to explicitly abort the program if strlen(input) is  
    // too long.  
    str[sizeof(str) - 1] = '\0';  
  
    // ...  
}
```



测试示例

```
import static org.junit.jupiter.api.Assertions.assertEquals;

import example.util.Calculator;

import org.junit.jupiter.api.Test;

class MyFirstJUnitJupiterTests {

    private final Calculator calculator = new Calculator();

    @Test
    void addition() {
        assertEquals(2, calculator.add(1, 1));
    }

}
```

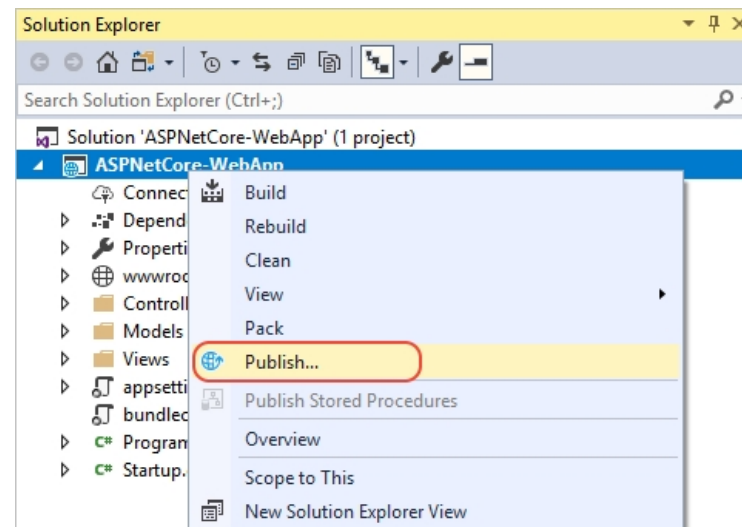
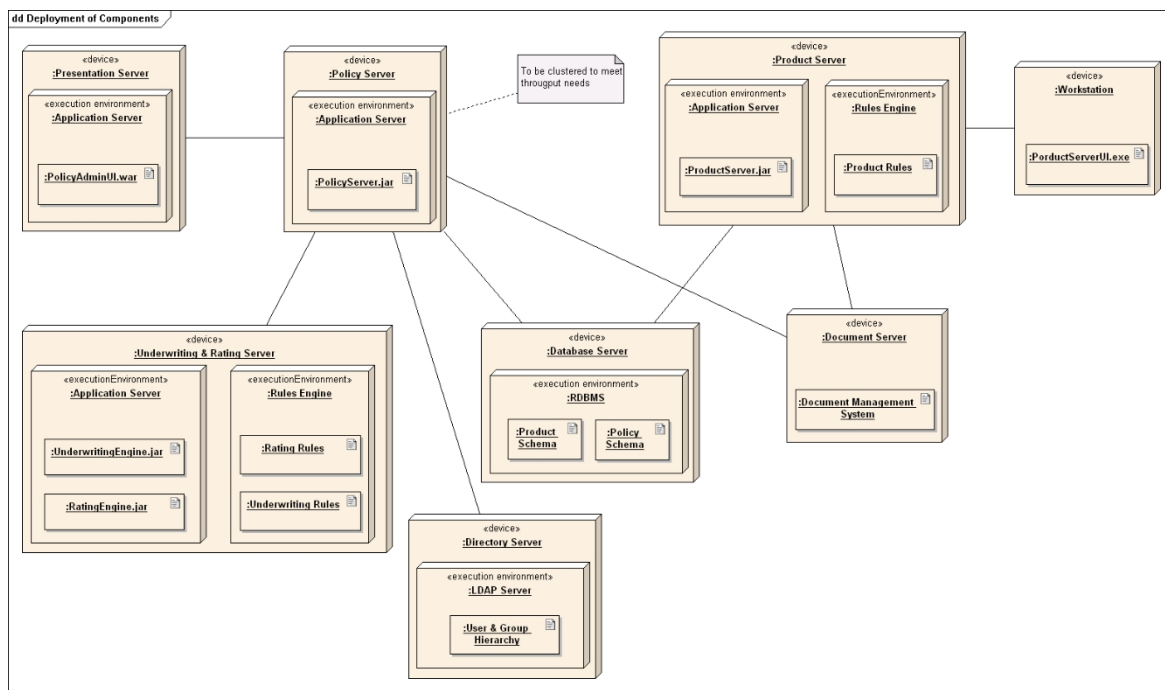
单元测试

单元测试：测试单个类或单个函数

集成测试：测试多个类或函数的交互
系统测试：以端到端方式测试从用户输入到系统输出的全过

程

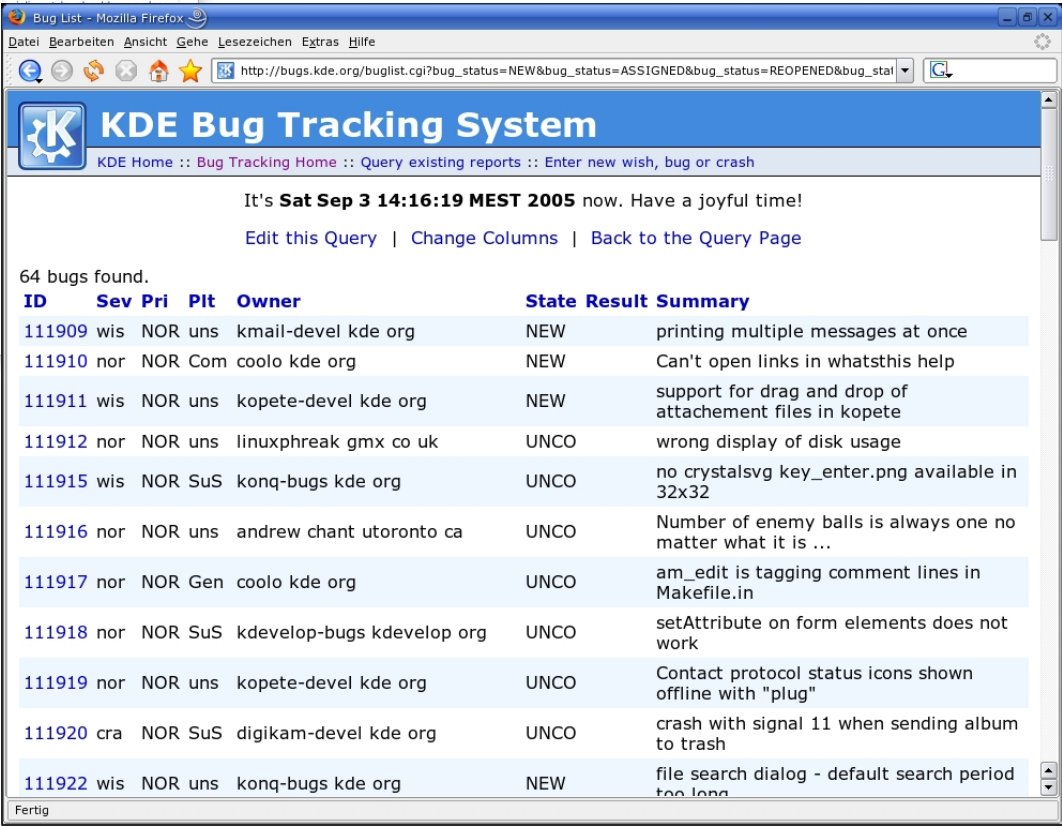
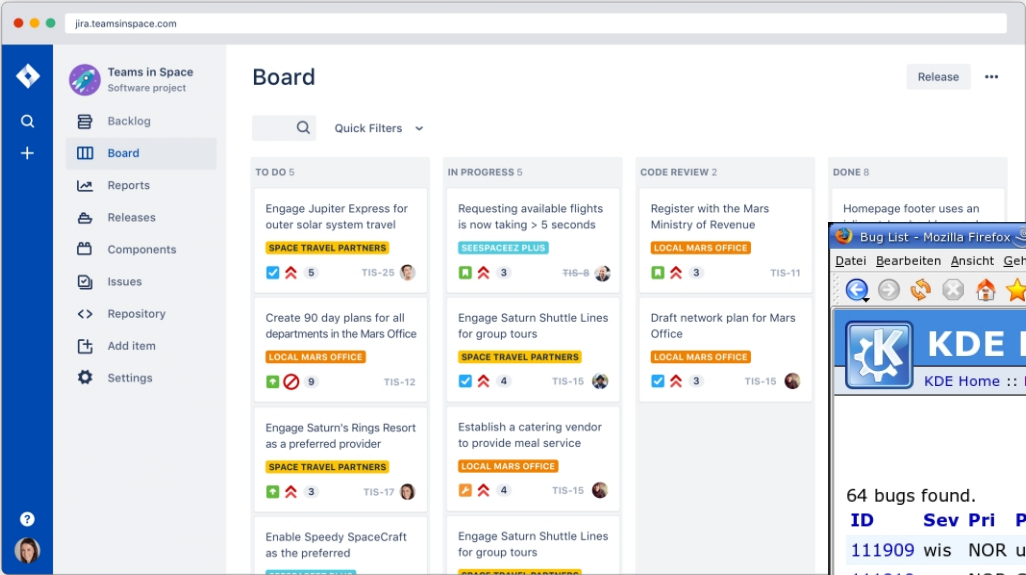
示例化软件部署



部署图：映射软件组件在硬件设备（或硬件虚拟化）上的运行

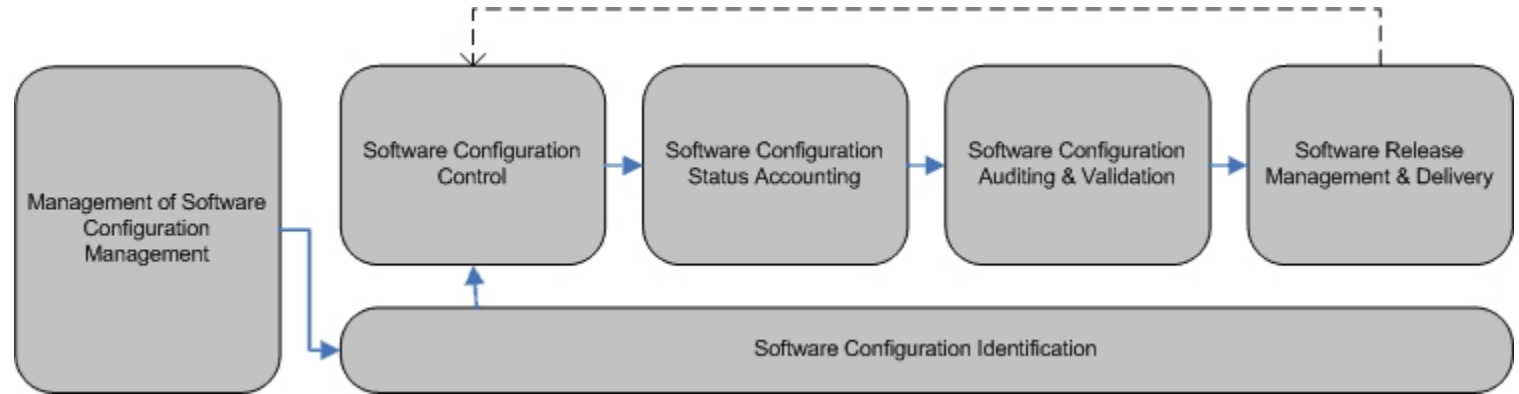
编写脚本或使用工具进行发布

示例化软件维护

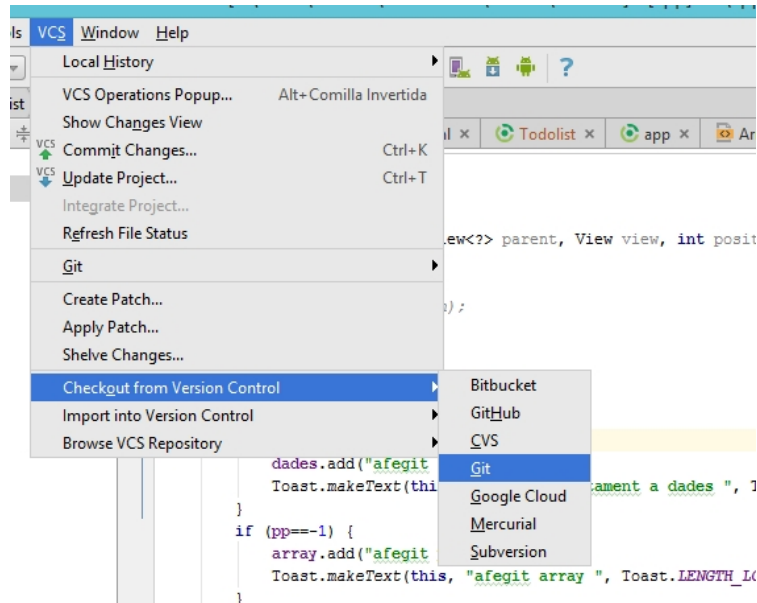


范例化配置管理

◆ 流程

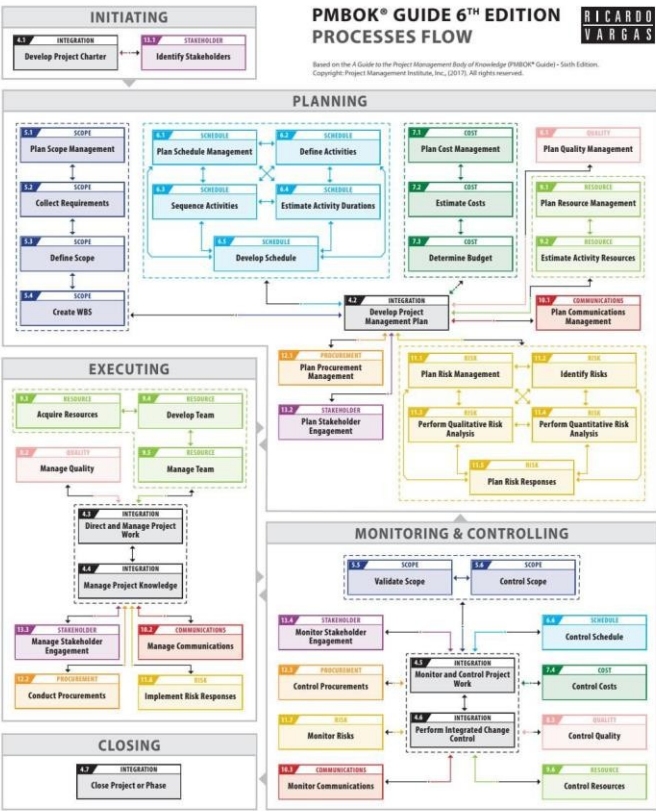


代码版本控制



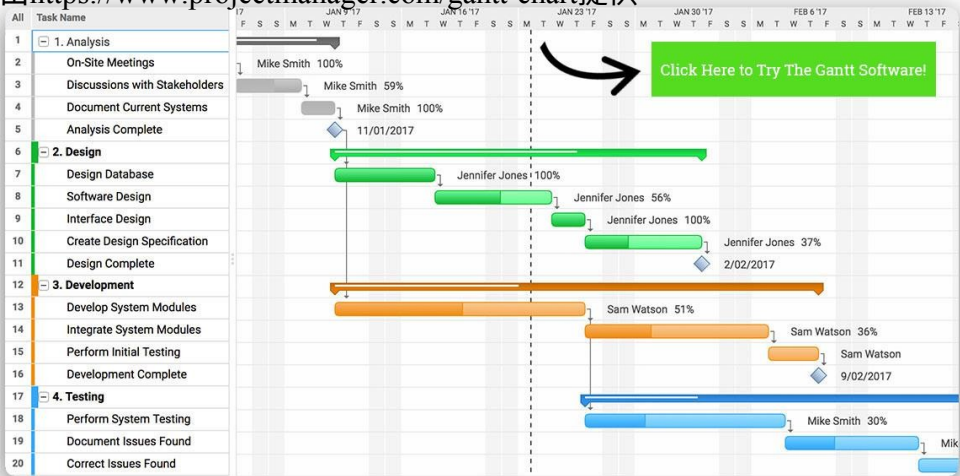
范例项目管理

PMBok



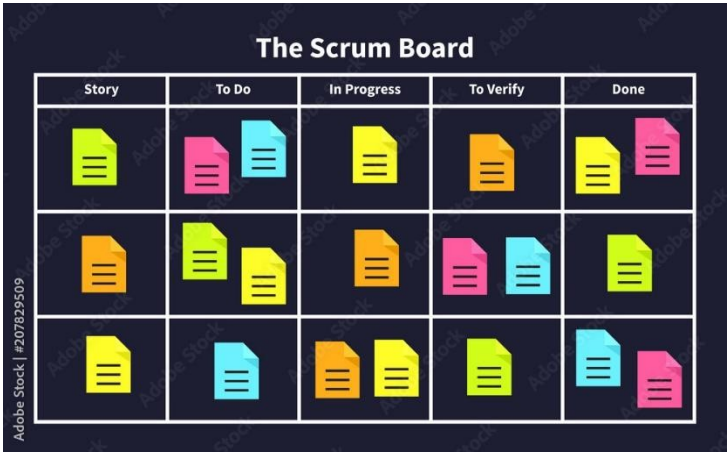
PMBOK® GUIDE 6TH EDITION – 49 PROJECT MANAGEMENT PROCESSES
Copyright © A Guide to the Project Management Body of Knowledge (PMBOK® Guide) – Sixth Edition, Project Management Institute, Inc. (2017). All rights reserved.
Explanatory Note: This flow does not replace the need for reading the PMBOK® Guide. The PMBOK® Guide contains a deep explanation of all processes including inputs, tools & techniques and outputs that are not listed in this flow.
Adaptation: Ricardo Vargas
Graphic Designer: Sérgio Alves Lima, Jordan
ricardo-vargas.com Download this processes flow at <http://rvarg.as/pmbok6en>
www.pmi.org

由<https://www.projectmanager.com/gantt-chart>提供



甘特图

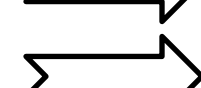
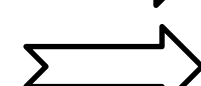
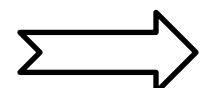
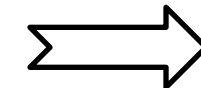
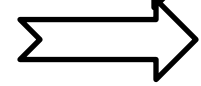
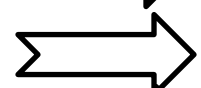
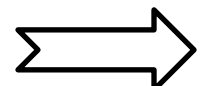
敏捷开发
任务板



SE 流程 — 技术

示例

- ◆ 收集用户需求
- ◆ 软件设计
- ◆ 编码/实现
- ◆ 测试
- ◆ 在用户站点部署软件
- ◆ 维护软件
- ◆ 配置管理
- ◆ 项目管理



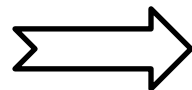
- ◆ 用例/用户故事/用例图/会议
- ◆ UML/模式/原则/...
- ◆ Java/C++/...
- ◆ 单元测试/调试器/...
- ◆ 独立软件/软件插件/应用程序/网络服务/...
- ◆ 缺陷报告/软件仓库
- ◆ 版本控制/变更管理软件
- ◆ 工作分解结构/

任务调度算法

SE 流程 — 交付成果

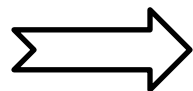
示例

◆ 用例/用户故事/用例图



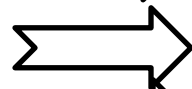
◆ 需求文档

◆ UML/模式/原则/...



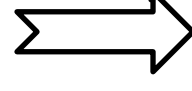
◆ 软件设计文档

◆ Java/C++/...



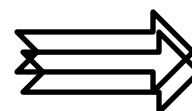
◆ 代码列表和测试脚本

◆ 单元测试/调试器/...



◆ 质量保证 (QA) 报告

◆ 软件插件/小程序架构

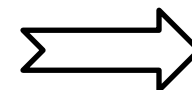


◆ 用户现场软件

◆ 错误报告/软件库

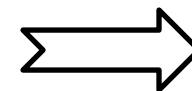
◆ 错误报告与软件发布

◆ 版本控制/变更管理软件



◆ 代码变更/补丁/变更历史报告

◆ 工作分解结构/任务



◆ 项目进度与状态

调度算法

追踪

SE流程 — 工具

示例

◆ 用例/用户故事/用例图	➡	◆ UML工具/需求建模工具/Word
◆ UML/模式/原则/...	➡	◆ UML工具/SonarQube
◆ Java / C++/...	➡	◆ Visual Studio/Eclipse
◆ 单元测试/调试器/...	➡	◆ JUnit/调试器/模糊测试器...
◆ 软件插件/应用程序/网络服务	➡	◆ 安装程序/插件框架
◆ 错误报告/软件库	➡	◆ Bug报告系统
◆ 版本控制/变更管理软件	➡	◆ Apache Subversion/Git
◆ 工作分解结构/任务	➡	◆ MS Project

SE流程 — 最佳实践

◆ 组织活动的不同流程定义了不同类型的软件开发流程

◆ 例如, [详见接下来的几张幻灯片]

- 瀑布模型
- V型瀑布模型
- 增量式软件开发模型
- 原型开发法
- 螺旋模型
- 统一过程 (UP)
- 敏捷方法

1960年代

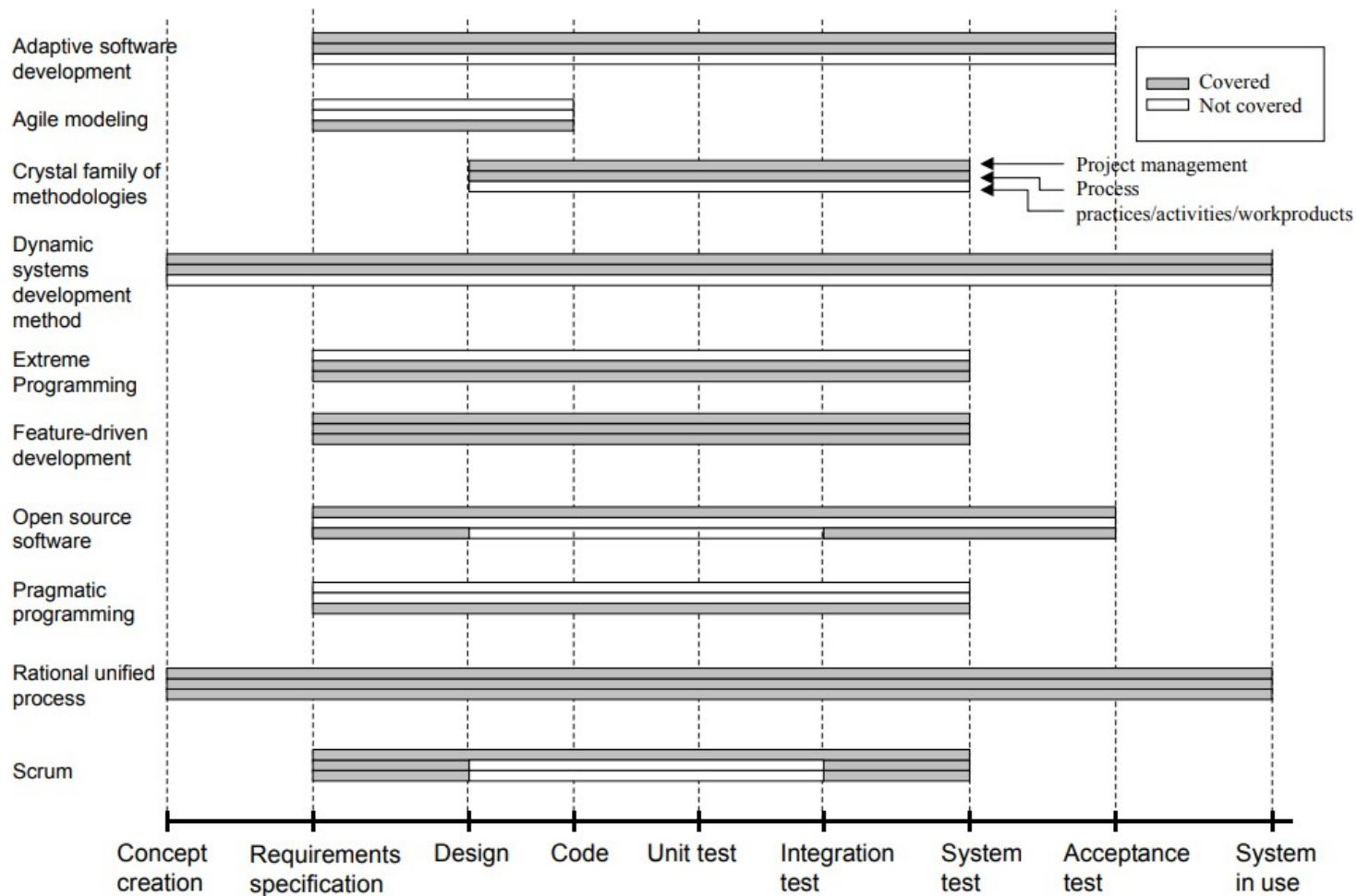
1980年代

2000年代

2010年代

多种敏捷方法

全部10种都是敏捷方法。



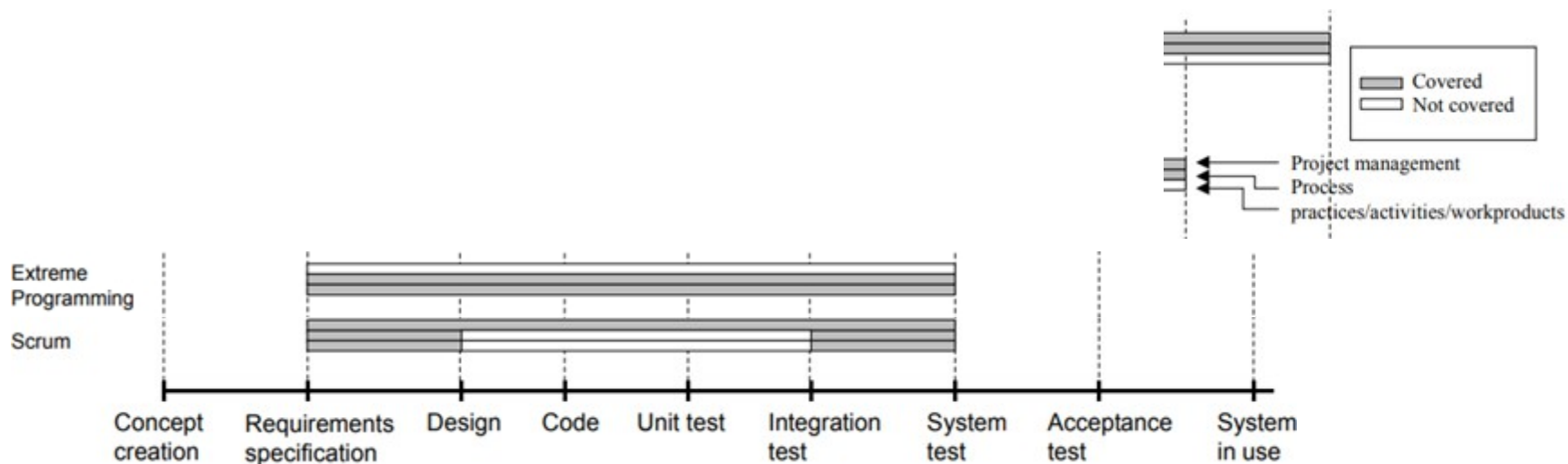
一分钟问答

- ◆ **问题1：** 在最后一页幻灯片中，哪些敏捷方法包含项目管理活动？
- ◆ 若某敏捷方法所需的特定工具、活动、流程步骤及中间产出更少，则该方法更轻量化。
- ◆ **问题2：** 是否存在既轻量化又包含项目管理的敏捷方法？

活动？请列举

极限编程 vs Scrum

- ◆ XP不设项目经理。
- ◆ Scrum包含项目管理（但流程比XP更简洁）及其他实践/活动/工作成果

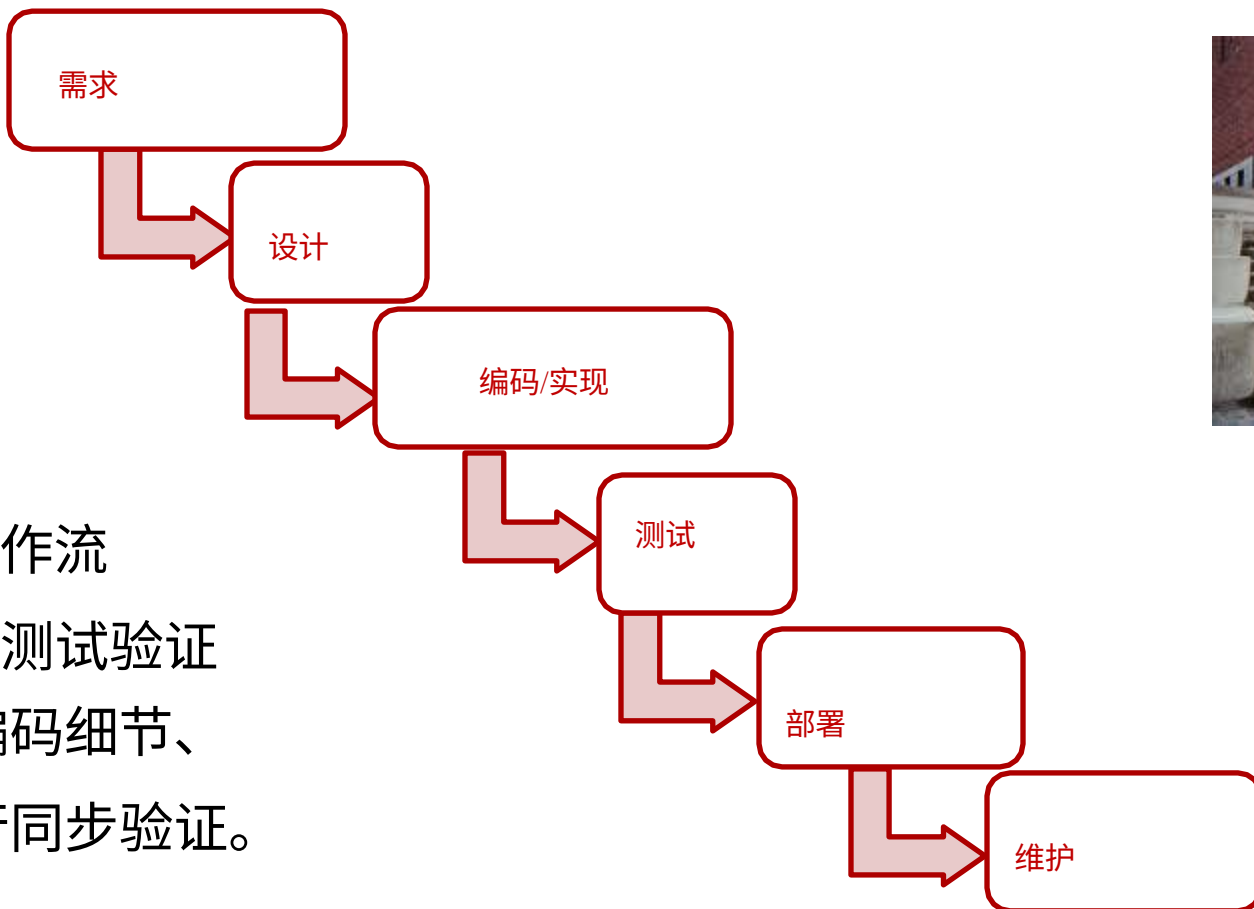


流程简览

模型

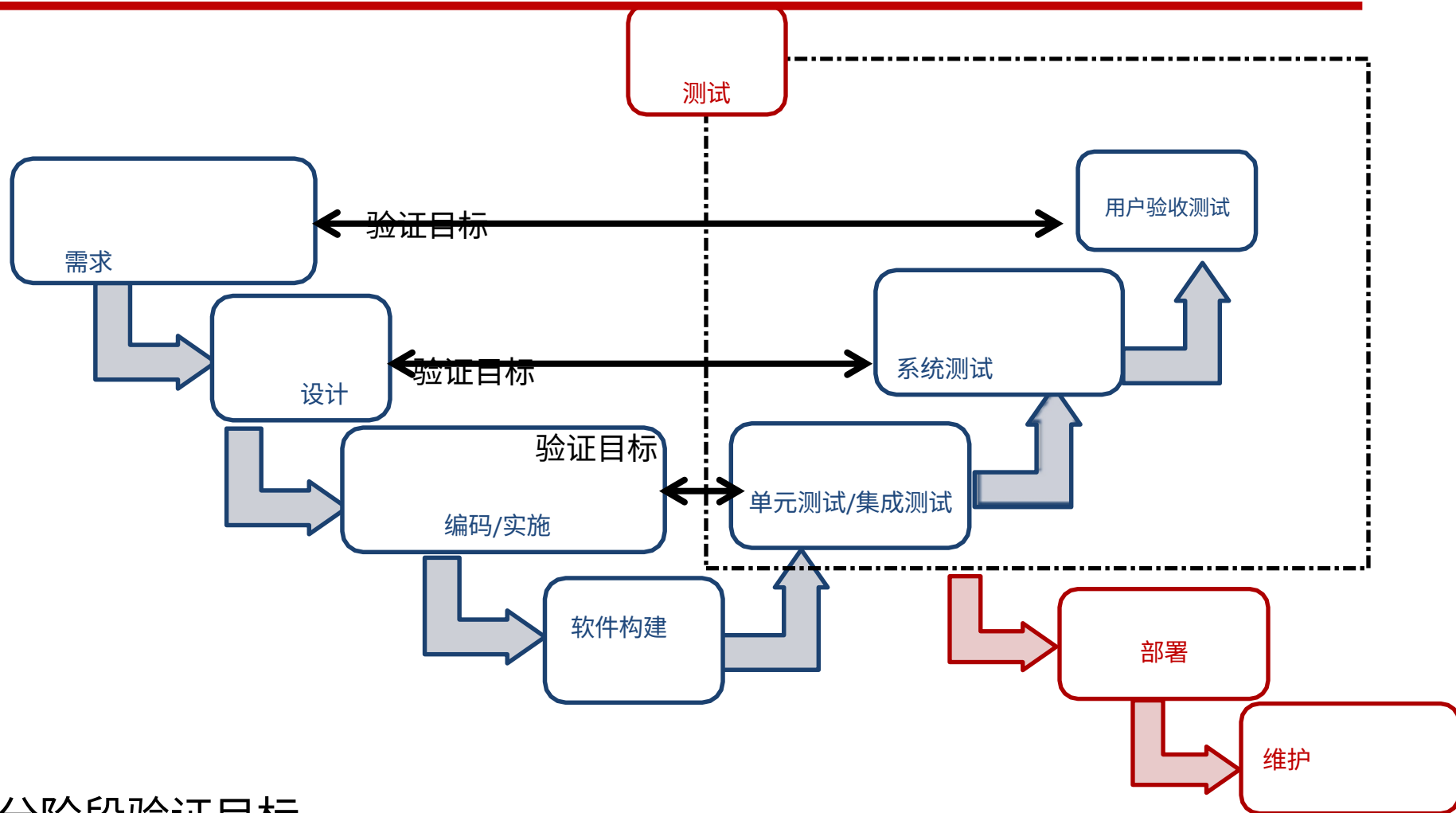
瀑布模型

- ◆ 在开始下一项活动之前，*必须完全完成*每项活动的全部交付成果。



- + 简单 workflow
- 需进行测试验证设计、编码细节、需求进行同步验证。

V-塑造瀑布模型



+ 分阶段验证目标

“瀑布模型”与“V型瀑布模型”的问题

1. 对活动阶段划分存在错误认知。

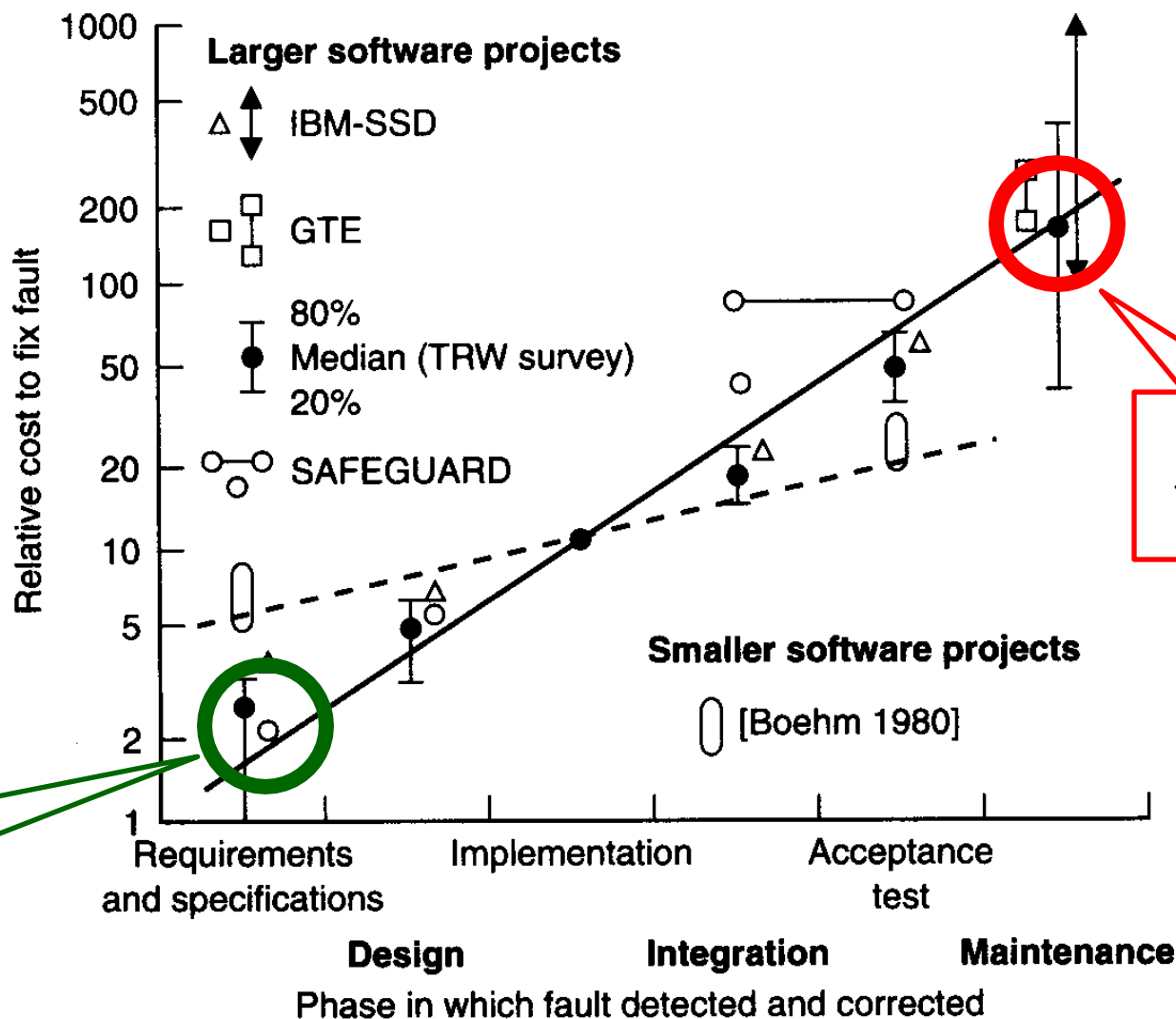
- 例如：在设计软件之前，我们能否完全厘清实际需求？
- 对于现实世界的问题而言，这是不切实际的。
- 纠正上游活动中的问题成本极高。[见下一张幻灯片]

2. 所有环节完成前，任何工作都算不上完成

- 例如：若项目止步于设计阶段，则无法向客户交付任何代码，但客户需要可执行的成果来推动自身业务！

经验教训:

早期修复缺陷的成本更低



采用瀑布模型的影响

我们宁愿在项目结束时拥有一个**功能逐步退化的**子系统

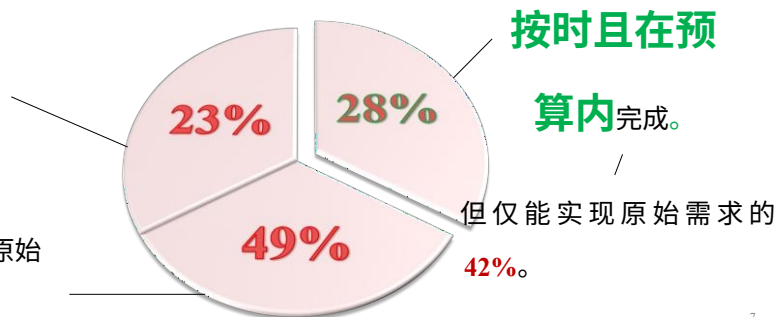
- 一个部分系统，一个质量较低的系统，也不愿最终一无所获！

可验证的**早期发布子**
系统成果应尽早验证
？ 如何实现？

事实：
软件开发项目

在
完成前。

成本超出原始
预算**89%**



当前趋势 (1)

- ◆ 某些阶段（活动）**可并行**开展
- ◆ 示例：并行开展技术开发与用户培训

- 开发与
每个并行轨道内的验证。
- 每个并行轨道都属于一个独立的子系统。
- 各并行轨道推进速度更快，从而更早修正缺陷→降低成本

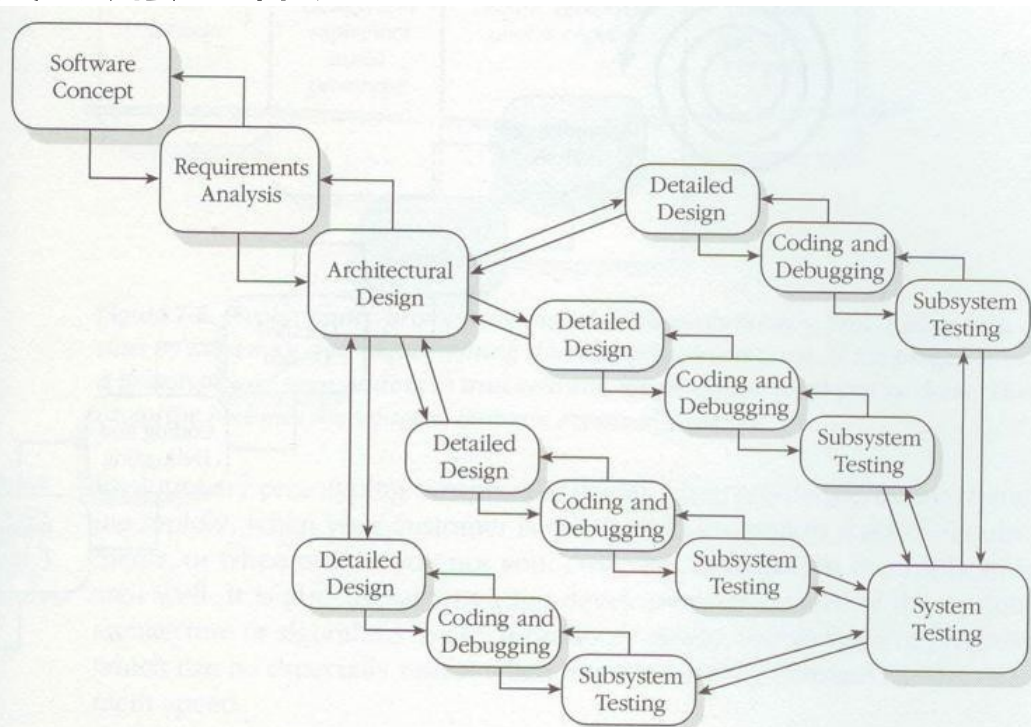
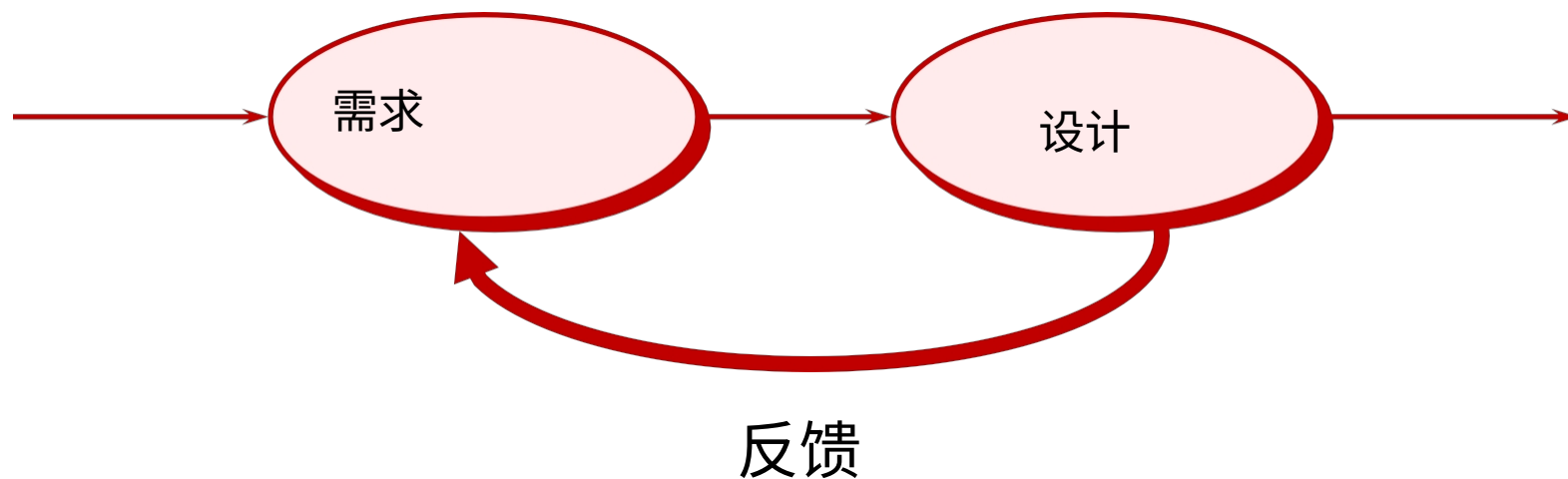


Figure 7-6. The waterfall model with subprojects. Careful planning can allow you to perform some of the waterfall's tasks in parallel.

当前趋势 (2)

◆ 向后迭代

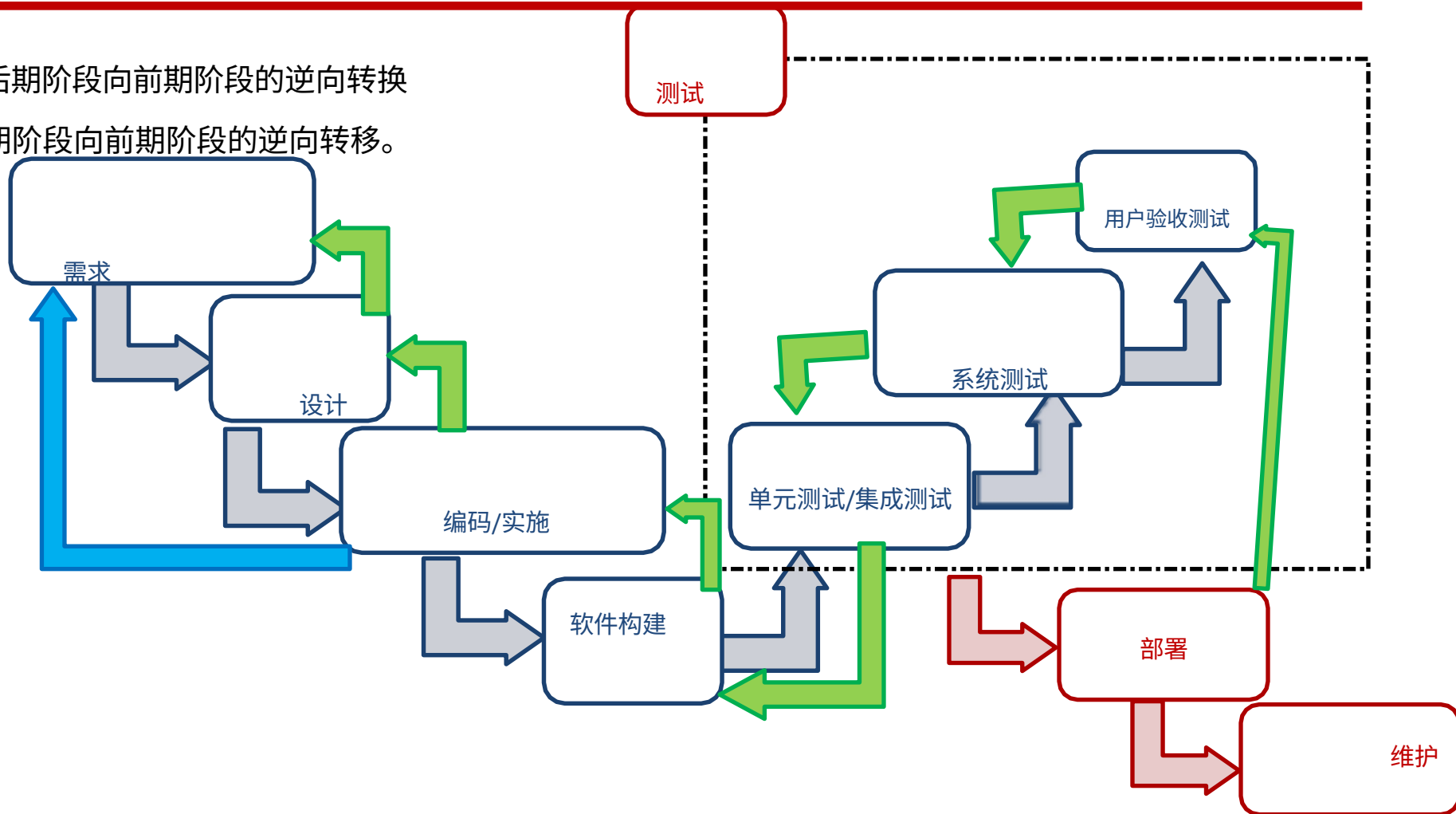
示例:



- 注意箭头方向。
- 必要时改进前期阶段
- 通过制定更精确的需求避免不必要的返工或设计缺陷

V型瀑布模型

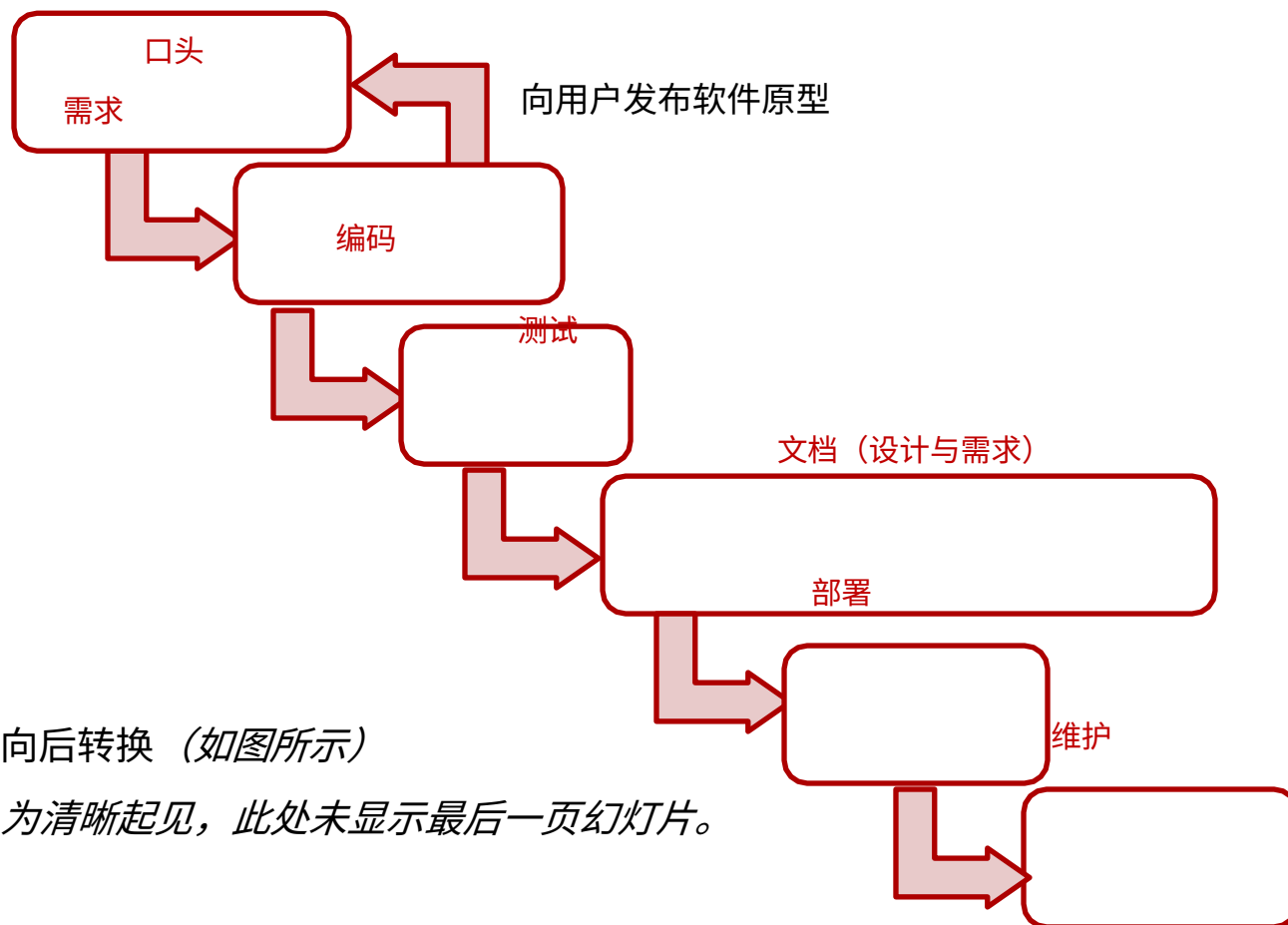
从后期阶段向前期阶段的逆向转换
后期阶段向前期阶段的逆向转移。



原型设计

原型制作：

+ 设计与需求文档前的代码



向后转换（如图所示）

为清晰起见，此处未显示最后一页幻灯片。

原型是最终产品的版本，可能存在缺陷，仅实现部分功能或仅包含界面

效果，让用户能够“看到”产品，从而比用文字和UML表达更轻松地指出他们的需求。

在正式进入软件设计阶段前，需逐步获取并验证用户/系统需求。

增量开发

◆ 除采用并行开发小规模子系统外，另一种方式是交付较小需求集并逐步扩展需求范围。

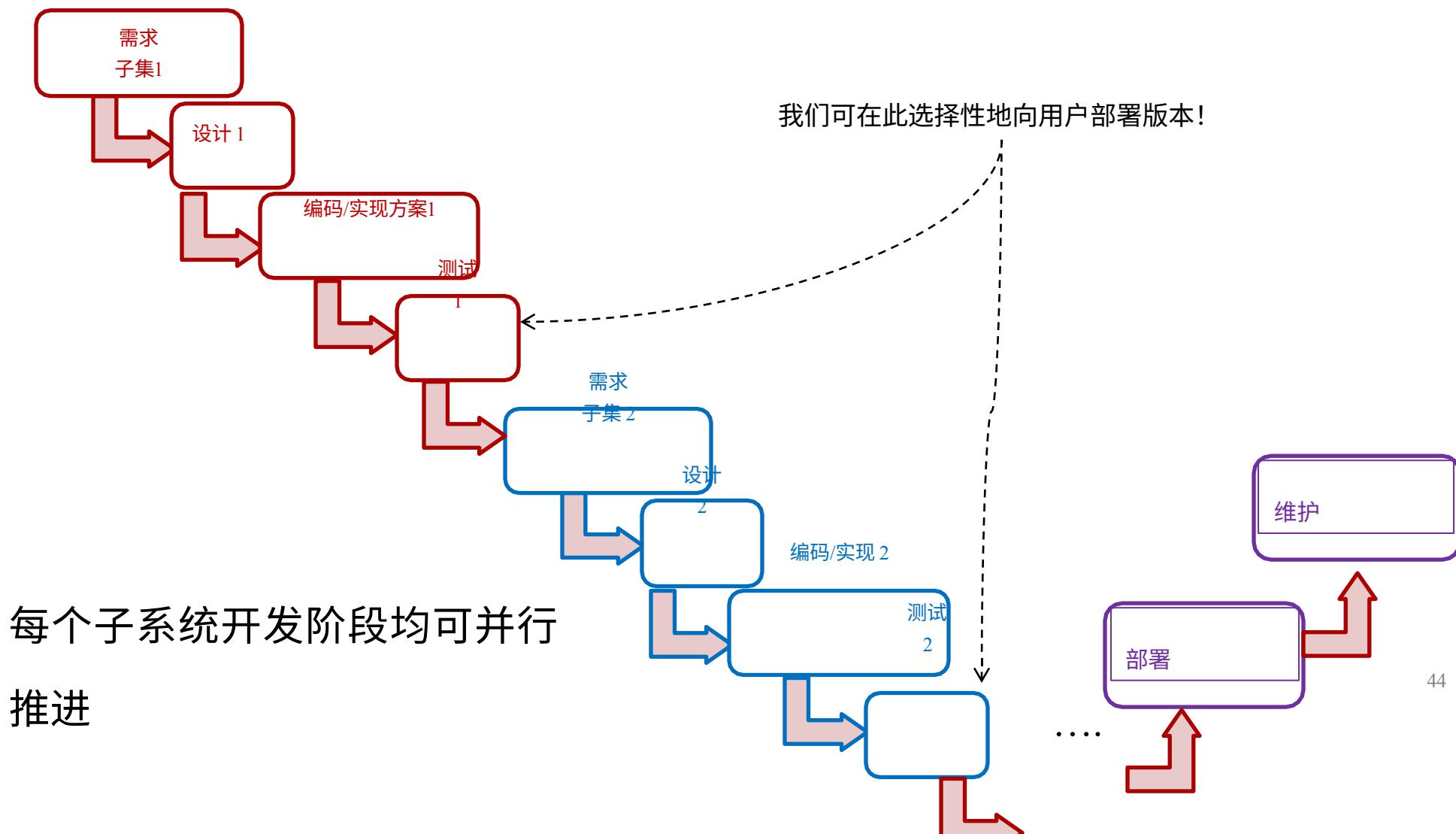
◆ 将需求集划分为子集。先将其中一个子集实现为程序版本v1，

在v1基础上实现另一子集作为版本v2，随后

在v2基础上实现第三个子集版本v3

如此循环.....

增量开发

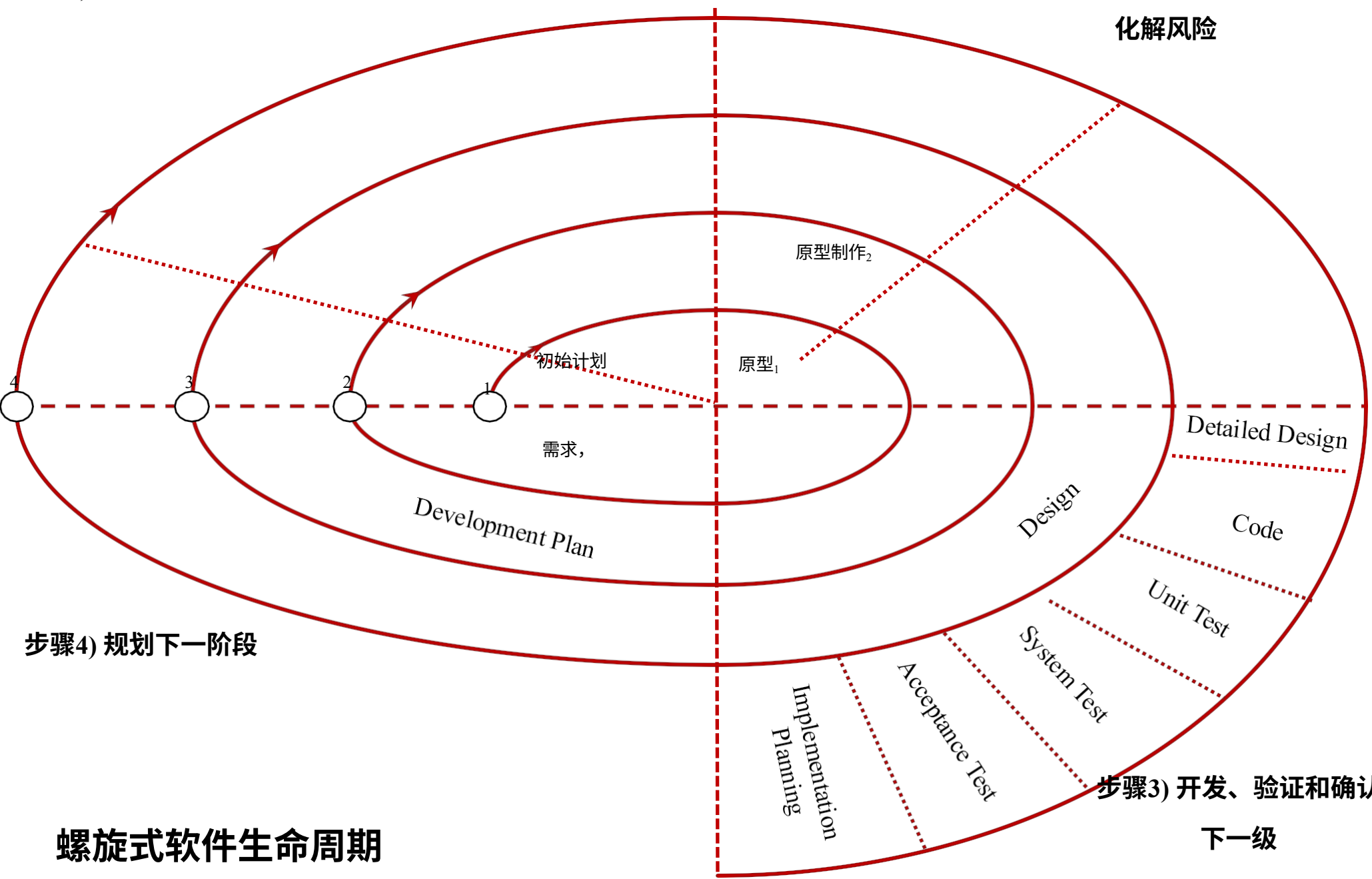


螺旋模型

- ◆ 兼具瀑布式、原型开发和增量开发的特质，但更具系统性。
- ◆ **基本思路**: 规划、制作原型、修订先前计划，通过一系列迭代直至充分掌握 **项目X**。

步骤1) 确定目标、备选方案及约束条件

步骤2) 评估备选方案、识别风险并
化解风险



软件过程改进

当前趋势

◆ 基于模型的软件工程

过程

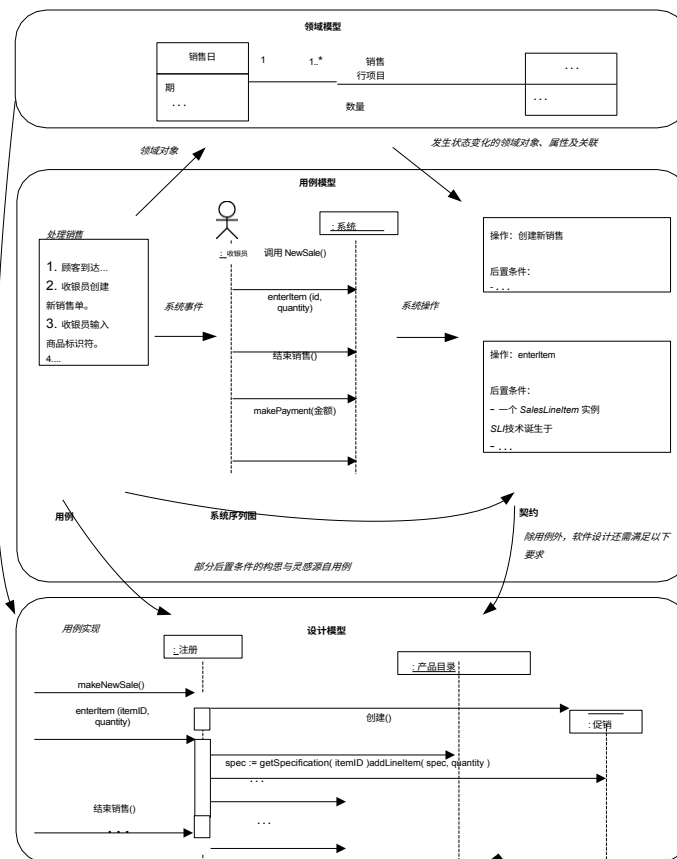
(各种
建模
技能如
技术)



交互

开发者

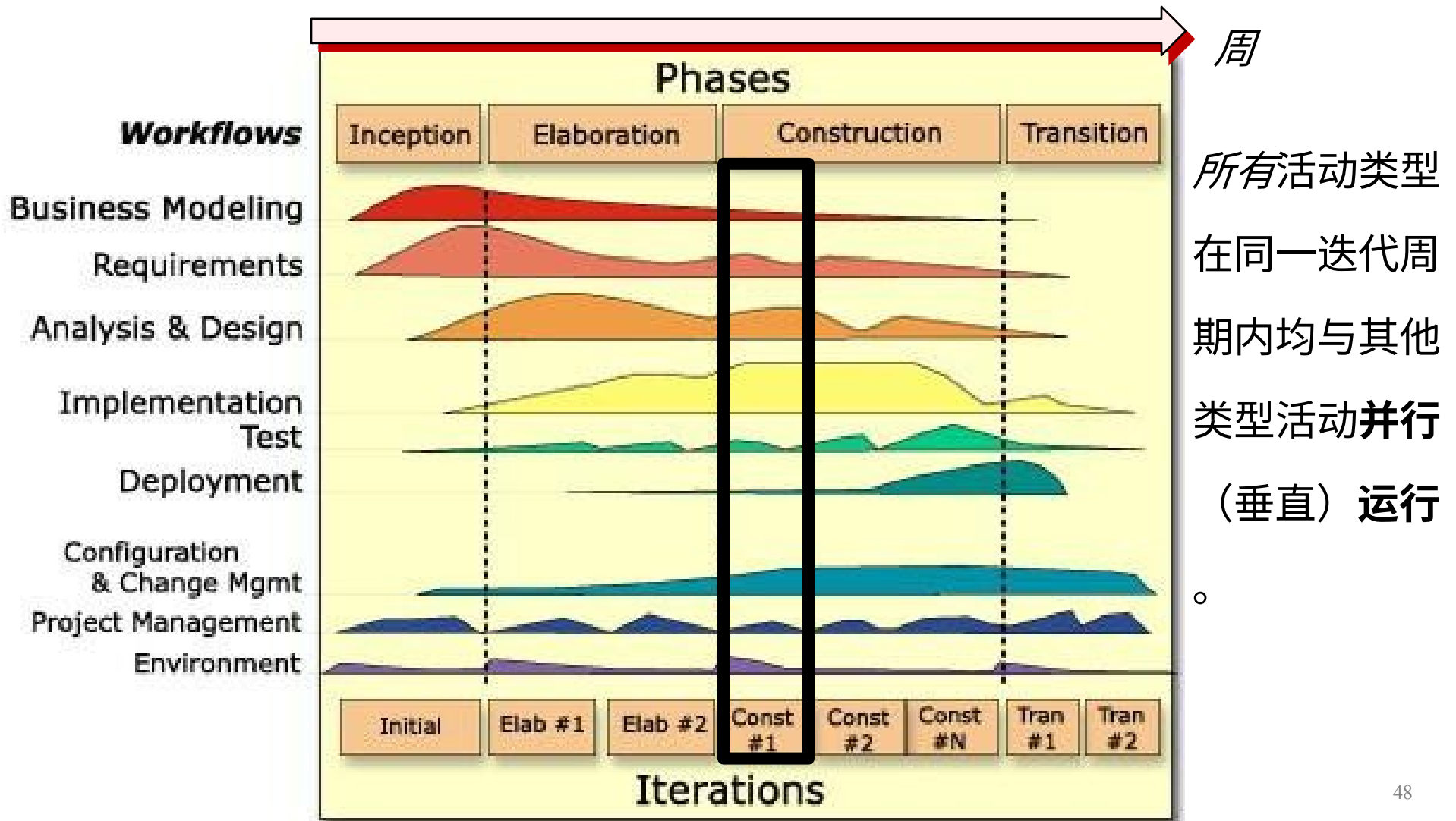
交互



构造

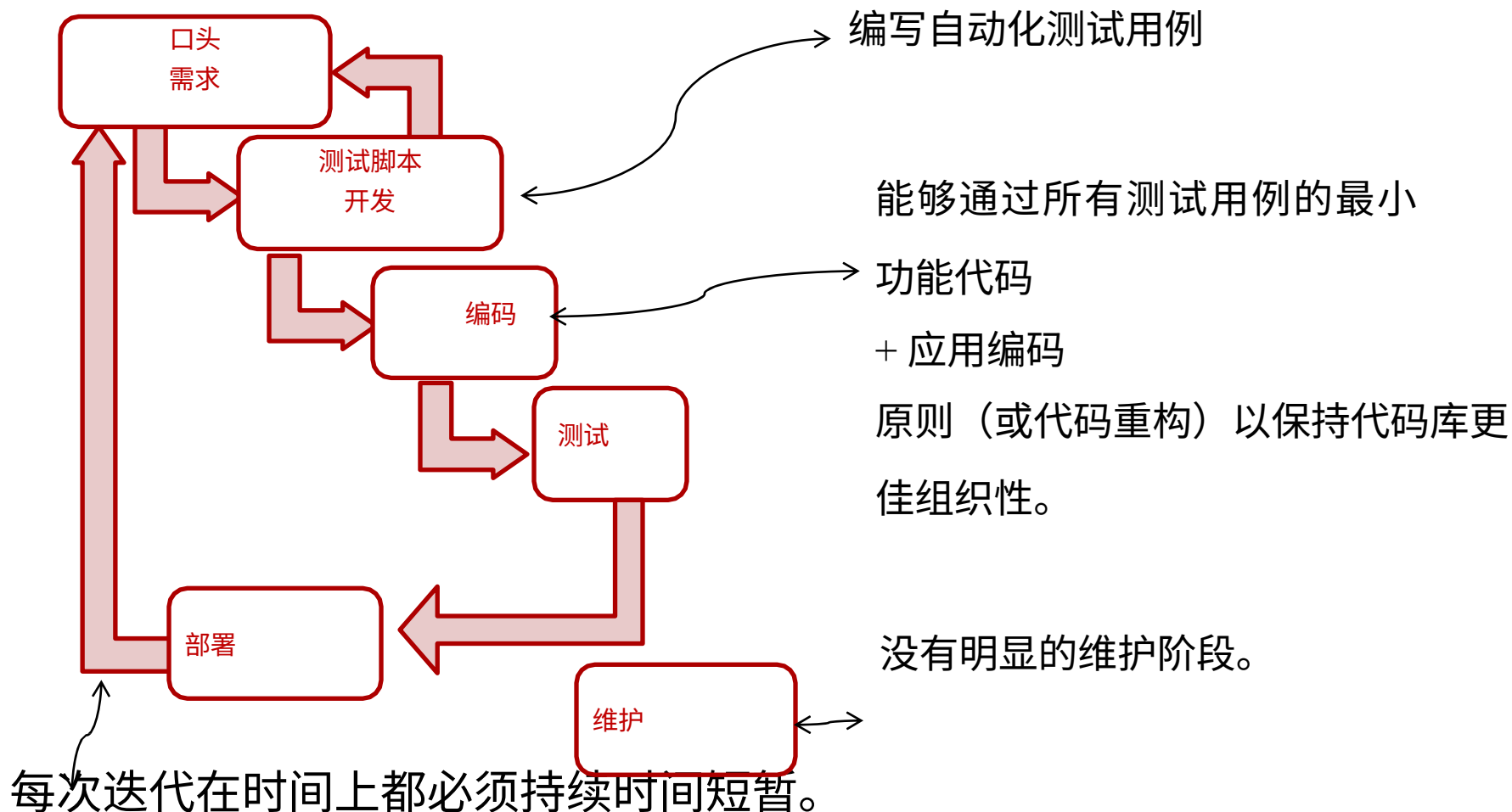


(Rational)统一过程



所有图表均为便于沟通的草图。

测试驱动开发理念



XP

◆ 极限编程（XP）包含五个阶段：

■ 规划

- 与用户会面，定义用户故事，并估算故事点。通过N次迭代规划发布（用于用户故事交付）

■ 设计

- 采用简洁统一的设计草图

■ 编码

- 应用极限编程实践（详见后续两张幻灯片）

■ 测试

- 针对每个用户故事执行自动化单元测试和验收测试

■ 反馈

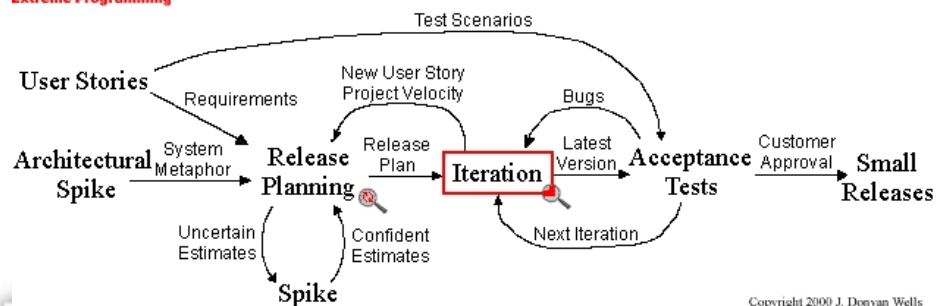
- 项目经理和用户共同确定交付用户故事的价值

实现

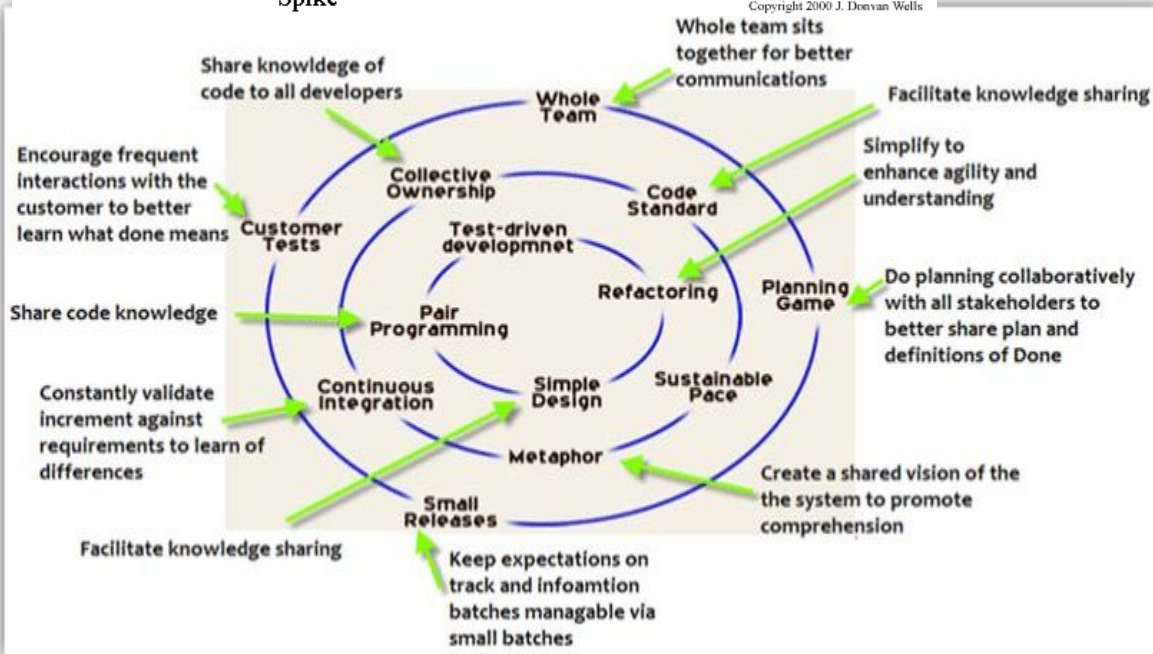
极限编程XP [一种敏捷方法]



Extreme Programming Project



Copyright 2000 J. Donovan Wells



◆ 重点:

- 收集反馈
- 拥抱变更
- 保持客户参与
- 短迭代周期
- 早期修复缺陷

XP实践



The Values of Extreme Programming

Extreme Programming Feedback: We will take every iteration seriously by delivering working software early and often then our values. XP isn't really a set of rules but rather a way to work in harmony with your personal and corporate values. Start with XP's values listed here then add your own by reflecting them in the changes you make to the rules.

Respect: Everyone gives and feels the respect they deserve as a valued team member.

Simplicity: We will do what is needed and asked for, but no more. This will maximize the value created for the investment made to date. We will take small simple steps to our goal and mitigate failures as they happen. We will create something we are proud of and maintain it long term for reasonable costs.

Courage: We will tell the truth about progress and estimates. We don't document excuses for failure because we plan to succeed. We don't fear anything because no one ever works alone. We will adapt to on everything from changes when ever they requirements to code. We will create the best solution to our problem that we can together.

Communication: Everyone is part of the team and we communicate face to face daily. We will work together on everything from changes when ever they requirements to code. We will create the best solution to our problem that we can together.

What lessons have we learned about implementing XP so far. ☺☺



The Rules of Extreme Programming

Planning

- User stories are written.
- Release planning creates the release schedule.
- Make frequent small releases.
- The project is divided into iterations.
- Iteration planning starts each iteration.

Managing

- Give the team a dedicated open work space.
- Set a sustainable pace.
- A stand up meeting starts each day.
- The Project Velocity is measured.
- Move people around.
- Fix XP when it breaks.

Designing

- Simplicity.
- Choose a system metaphor.
- Use CRC cards for design sessions.
- Create spike solutions to reduce risk.
- No functionality is added early.
- Refactor whenever and wherever possible.



Coding

- The customer is always available.
- Code must be written to agreed standards.
- Code the unit test first.
- All production code is pair programmed.
- Only one pair integrates code at a time.
- Integrate often.
- Set up a dedicated integration computer.
- Use collective ownership.

Testing

- All code must have unit tests.
- All code must pass all unit tests before it can be released.
- When a bug is found tests are created.
- Acceptance tests are run often and the score is published.

XP指南

- ◆ **规划：**程序员估算实现用户故事所需的工作量，客户决定发布范围与时间节点
- ◆ **小规模发布：**每月一次，或每日进行小幅修复
- ◆ **隐喻：**通过描述系统运作方式，共享故事引导所有开发工作
- ◆ **简约设计：**采用最简方案
- ◆ **测试：**代码编写前先实现测试。客户编写功能测试。
- ◆ **重构：**频繁进行重构
- ◆ **结对编程：**两人共用一台计算机编写代码。
- ◆ **集体所有权：**任何人都可随时修改代码的任何部分
- ◆ **持续集成：**代码完成后立即集成到项目代码库中。
- ◆ **现场客户：**客户全程参与
- ◆ **编码规范：**严格执行。
- ◆ **开放式办公环境：**优先选择带小型隔间的大型空间
- ◆ **基本规则：**团队有自己的规则，但规则可变，所有人都需遵守

◆ **每周40小时工作制：**连续两周内不加班

◆ **基本规则：**团队有自己的规则，但规则可变，所有人都需遵守

常见项目管理要素

- ◆ 为每个用户故事估算*故事点*
 - 为不确定性高的用户故事预留额外研究时间
- ◆ 在XP中挑选一组用户故事进入下个迭代（相当于Scrum中的下个冲刺）
- ◆ 在每个周期（XP迭代或Scrum冲刺）内举行（每日）站立会议
 - 超短会议（总计15分钟）
 - [3分钟] 全员汇报
 - (1) 昨日完成事项
 - (2) 今日计划完成事项
 - (3) 遇到的阻碍事项。此项内容可由其他团队成员协助解决。
- ◆ 衡量*项目速度*
- ◆ 在每个周期结束时，进行一次评审（向用户展示该周期内完成的可交付成果）并开展周期自省

用户故事

◆ 用户故事采用以下格式：

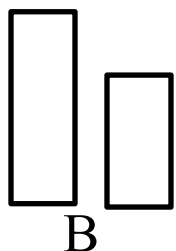
- 作为<角色>，我需要<操作>，以便实现<目标>。

- 角色可通过需求工程识别
- 操作应具可操作性
- 目标应可衡量且可量化

例如（针对AIMS系统）：作为城大学生，我希望通过在AIMS系统提交申请，下载证明学生身份的证明文件，以便向香港特别行政区入境事务处申请学生签证延期。

- ## ◆ 用户故事通常包含更多信息，例如用例、图示草图、用户注释、数据以及现有系统的报告，以阐明用户故事的背景和范围。

相对故事点估算



◆ 很难估计条形图 A 是否较长以及条形图 A 的长度。

◆ 但要说A比B长就容易得多。

◆ 故事点是一个数值。

■ 例如，斐波那契数列：1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, ...

■ 例如：T恤尺码：XXS、XS、S、M、L、XL、XXL

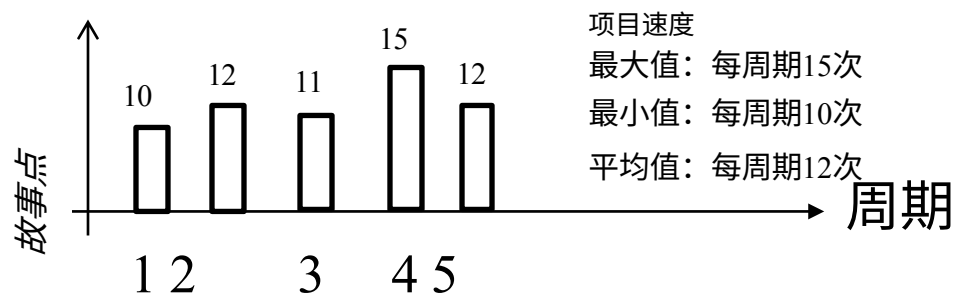
◆ 选取一个或多个用户故事作为基准。整个团队为每个此类用户故事商定一个故事点值，以表示实现该用户故事所需的预期工作量，同时考虑不确定性和实现复杂性。

■ 例如：若B故事点定为3，团队认为A

A比B大，但差距不大，可为A分配5个故事点。

项目速度

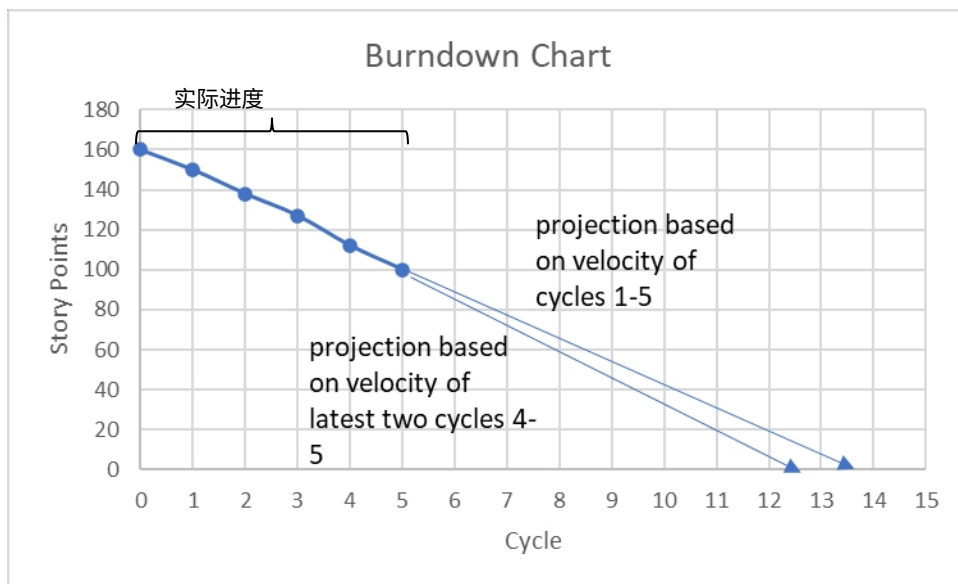
- ◆ 用户故事在通过周期评审（由用户和产品负责人执行）后即视为在本周期内实现
- ◆ 敏捷项目以一系列周期（XP迭代或Scrum冲刺）的形式组织。
- ◆ 每个周期实现一组带有分配故事点的用户故事。
- ◆ 若用户故事规模过大无法纳入单个周期，应将其拆解为多个用户故事。
- ◆ 该周期交付的故事点总数绘制成图。



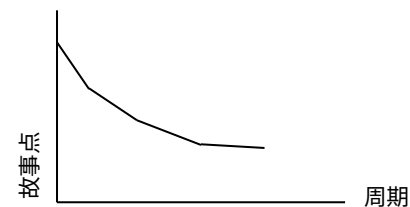
若剩余故事点数为100，则项目需再进
 $\lceil 100/12 \rceil$
= 9个周期才能完成

燃尽图

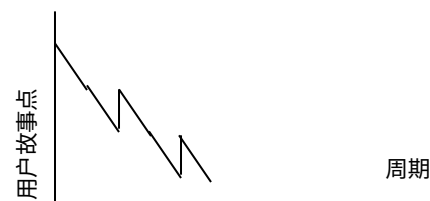
周期	故事点
0	160
1	150
2	138
3	127
4	112
5	100



问题！项目无法逐步推进



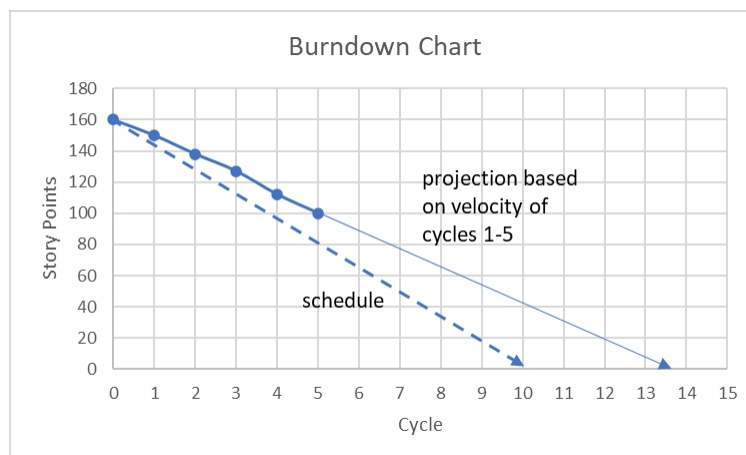
哎呀！项目中新增用户故事或原始估算的故事点数被低估



项目速度是连续多个周期中斜率的累积值。

计划值与实际值

- ◆ 在制定发布计划或冲刺计划时，项目经理需要一个日程安排。
- ◆ 以我们的课程项目为例（必须在第12周结束前交付）。
 - 假设项目于第2周启动，每个周期为1周。最后一个周期用于项目收尾工作，包括撰写报告、制作海报或视频等。
 - 至第7周结束时（完成5个周期），项目出现延误。
 - 项目经理需在每个周期结束后采取行动：例如精简部分用户故事并要求团队在每个周期交付更多故事点



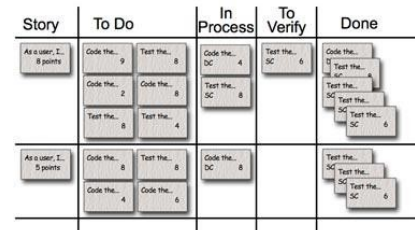
Scrum

http://en.wikipedia.org/wiki/Scrum_ (开发)

- ◆ 是一种敏捷开发流程
- ◆ 堪称当今最广泛采用的开发流程
- ◆ 通过每日及定期一小时会议驱动
 - 梳理使用场景/用户故事，筛选出最高优先级的待办事项切片，经客户确认后纳入下个迭代周期（冲刺）完成
 - 与客户共同商定当前冲刺目标，对待完成功能进行优先级排序并分解为具体任务
 - 定期召开简短的计划会议和评审会议，针对已完成和未按时完成的任务进行评估，并据此调整待办事项清单
- ◆ 客户随时可能改变需求
 - 接受需求无法被完全理解或定义的事实，
 - 专注于最大化团队快速交付的能力，并响应选定待办事项的能力。

Scrum

任务看板



- ◆ 产品待办事项列表 = 用户故事及因某些用户故事完成后产生的开发任务（例如后期周期发现的缺陷）
- ◆ 对产品待办事项进行优先级排序
- ◆ 假设一个周期 = 两周（因此项目最多包含六个冲刺）
- ◆ 产品待办事项列表中优先级较高的子集被视为当前迭代待办事项列表
 - 冲刺待办事项的规模取决于团队基于历史经验，在单个周期内能够交付的故事点总数。
- ◆ 通过每日站会，每位成员从冲刺待办事项中挑选一项任务，目标是在当前周期内交付用户故事（可视化**为任务看板**）。
 - 当用户故事启动时，所需交付的任务均置于看板的"待办"列。
 - 当任务开始处理时，将其移至"进行中"列。
 - 当任务实施完成后，将其移至“待验证”列（该列通常与“进行中”列合并）
- ◆ 冲刺结束时，进行冲刺评审
 - 产品负责人/用户验证用户故事是否交付。若确认无误，则将任务移至"已完成"列。
- ◆ 同时进行冲刺回顾（反思哪些做法有效推动了进展）

以及需要改进的内容以减少或避免阻碍因素，从而提高生产力等)

Scrum工具与会议

<https://www.mountangoatsoftware.com/agile/scrum/scrum-tools>

- ◆ **产品**待办事项列表包含最终产品所需的功能清单、待修复的缺陷、用于搭建和维护开发环境及用户站点的技术工作，以及项目需掌握的知识（例如学习使用新框架）。
- ◆ **发布燃尽图**用于追踪项目进度，每完成一个冲刺周期即更新图表数据。团队可自由选择进度衡量单位。
- ◆ **冲刺待办事项列表**包含冲刺期间需完成的任务清单，每日更新一次。
- ◆ **任务**看板是团队成员在冲刺期间可共同使用和添加的工具，以可视化形式呈现所有任务及其完成阶段。通常包含用户故事、待办事项、进行中、待验证和已完成等列。部分团队还会添加燃尽图、备注和测试项。
可将看板悬挂于墙面或数字化呈现。
- ◆ **会议**：产品负责人带着优先级排序的敏捷产品待办事项清单出席冲刺计划会议，向团队说明重点事项。团队随后确定可在即将开始的冲刺周期内完成的任务，并将产品待办事项清单中的条目移至冲刺待办事项清单。在此过程中，他们将每个Scrum产品待办事项细化为一项或多项冲刺待办事项任务。

Scrum工具

<https://www.mountangoatsoftware.com/agile/scrum/scrum-tools>

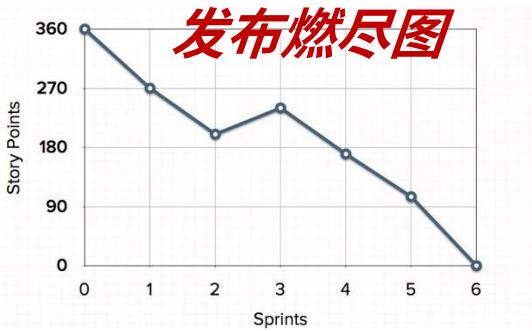
<https://www.mountangoatsoftware.com/agile/scrum/scrum-工具/产品待办事项列表/示例>

Profiles

- As a site member, I want to describe myself on my own page in a semi-structured way so that others can learn about me. That is, I can fill in predefined fields, but also have room for a free-text field or two. (It would be nice to let this free text be HTML or similar.)
- As a site member, I can fill out an application to become a Certified Scrum Practitioner so that I can earn that designation. [Note: Certified Scrum Practitioner was the initial name of what became known as Certified Scrum Professional.]
- As a Practitioner, I want my profile page to include additional details about me (i.e. some of the

任务看板

Story	To Do		In Process	To Verify	Done
As a user, I... 8 points	Code the... 9	Test the... 8	Code the... DC 4	Test the... SC 6	Code the... SC 8 Test the... SC 6 Test the... SC 6 Test the... SC 6
	Code the... 2	Code the... 8			
	Test the... 8	Test the... 4			
As a user, I... 5 points	Code the... 8	Test the... 8	Code the... DC 8		Test the... SC 8 Test the... SC 6 Test the... SC 6
	Code the... 4	Code the... 6			



冲刺待办事项

完成任务所需剩余工时

USER STORY	TASKS	DAY 1	DAY 2	DAY 3	DAY 4	DAY 5	...
As a member, I can read profiles of other members so that I can find someone to date.	Code the...	8	4	8	0		
	Design the...	16	12	10	4		
	Meet with Mary about...	8	16	16	11		
	Design the UI	12	6	0	0		
As a member, I can update my billing information	Automate test...	4	4	1	0		
	Code the other...	8	8	8	8		
	Update security tests	6	6	4	0		
	Design a solution to...	12	6	0	0		
	Write a test plan	8	8	4	0		
	Automate tests...	12	12	10	6		
	Code the...	8	8	8	4		

小时

天

和：102 86 69 33

Scrum角色

- ◆ **Scrum Master** 是团队的教练，帮助 Scrum 实践者实现最高水平的绩效。
 - 该角色不负责团队的日常指导，也不向个人分配任务。
 - 在许多项目中，该角色由项目经理兼任。
- ◆ **产品负责人**负责在Scrum开发过程中对待办事项列表进行优先级排序。
- ◆ **Scrum开发团队**整体。

敏捷方法比较

极限编程

Method name	Key points	Special features	Identified shortcomings
	the method development.	“visionary” and “advisor”.	method.
XP	Customer driven development, small teams, daily builds	Refactoring – the ongoing redesign of the system to improve its performance and responsiveness to change.	While individual practices are suitable for many situations, overall view & management practices are given less attention.
FDD	Five-step process, object-oriented component (i.e., feature) based development. Very short iterations: from hours to 2 weeks.	Method simplicity, design and implement the system by features, object modeling.	FDD focuses only on design and implementation. Needs other supporting approaches.
OSS	Volunteer based, distributed development, often the problem domain is more of a challenge than a commercial undertaking.	Licensing practice; source code freely available to all parties.	OSS is not a method itself; ability to transform the OSS community principles to commercial software development.
PP	Emphasis on pragmatism, theory of programming is of less importance, high level of automation in all aspects of	Concrete and empirically validated tips and hints, i.e., a pragmatic approach to software	PP focuses on important individual practices. However, it is not a method through which a system can be

Method name	Key points	Special features	Identified shortcomings
	programming.	development.	developed.
RUP	Complete SW development model including tool support. Activity driven role assignment.	Business modeling, tool family support.	RUP has no limitations in the scope of use. A description how to tailor, in specific, to changing needs is missing.
Scrum	Independent, small, self-organizing development teams, 30-day release cycles.	Enforce a paradigm shift from the “defined and repeatable” to the “new product development view of Scrum.”	While Scrum details in specific how to manage the 30-day release cycle, the integration and acceptance tests are not detailed.

Scrum

首页阅读

◆ 阅读本参考资料的以下章节：

<https://arxiv.org/ftp/arxiv/papers/1709/1709.08439.pdf>

■ 第3.2节 Scrum

◆ 阅读Scrum中Scrum Master与产品负责人的区别

- <https://www.yodiz.com/blog/scrum-master-vs-product-owner-differences-in-skills-duties-and-responsibilities-agile-methodology/>

参考文献

1. 若昂·M·费尔南德斯与索尼娅·M·索萨。2010。《PlayScrum——学习Scrum敏捷方法的卡牌游戏》。载于《2010年第二届严肃应用游戏与虚拟世界国际会议论文集》（VS-GAMES '10）。IEEE计算机学会，美国华盛顿特区，52-59页。DOI: <https://doi.org/10.1109/VS-GAMES.2010.24>
 - 包含Scrum的简要概述
2. YouTube视频
 - https://www.youtube.com/results?search_query=scrum